HPDP Learning Guide

The Hyperspectral Payload Data Processor from the basics to the advanced details: what it is, what it solves and how it works

Document	HPDP-GDE-001, revision A
Date	2026-10-21
Prepared for	Shubham Draveriya
System	Hyperspectral Payload Data Processor (HPDP) on the AMD ZCU106 board; final builds m7h (512 pixels) and m7i_1024b (1024 pixels)
Language	ASD-STE100 Simplified Technical English (STE)
Level	From no special knowledge (Part A) to the full design (Part C)
Related documents	SPEC.md (specification v1.0), docs/project_report.md, HPDP-BRF-001 (project brief), HPDP-PRC-001 (process description), HPDP-RPT-001 (results report), docs/m0 to m7 (milestone notes)

This guide teaches what the HPDP is, which problems it solves and how each part of it operates. It starts with the basic ideas and ends with the design details, the test results and the procedures.

Contents

Part A The basics . . . 5
1 About this guide . . . 5
1.1 Purpose . . . 5
1.2 How to read this guide . . . 5
1.3 Writing rules . . . 5
1.4 Warnings, cautions and notes . . . 5
2 The HPDP in one page . . . 6
2.1 What the HPDP is . . . 6
2.2 The four problems that the HPDP solves . . . 6
2.3 What the HPDP is not . . . 6
3 Basic ideas that you must know . . . 7
3.1 Bits, bytes and samples . . . 7
3.2 Data rate . . . 7
3.3 Clock and clock cycle . . . 7
3.4 Hyperspectral images . . . 7
3.5 Pushbroom imaging . . . 8
3.6 Streams and blocks . . . 8
3.7 Standards and CCSDS . . . 8
4 The hardware . . . 9
4.1 What an FPGA is . . . 9
4.2 The Zynq device: PS and PL . . . 9
4.3 The ZCU106 board . . . 10
4.4 SFP modules and the optical fiber . . . 10
5 The data chain at a glance . . . 11
5.1 The blocks . . . 11
5.2 The journey of one sample . . . 11
5.3 The satellite side and the ground side . . . 12
Part B How each block works . . . 13
6 The data budget . . . 13
6.1 Why the budget is the start of the design . . . 13
6.2 The calculation . . . 13
6.3 The proof that raw data does not fit . . . 13
7 The sensor emulator . . . 14
7.1 What it does . . . 14
7.2 How it operates . . . 14
7.3 It never waits . . . 14
7.4 Other functions . . . 14
7.5 Measured results . . . 15
8 Radiometric correction . . . 15
8.1 Why a detector needs correction . . . 15
8.2 The formula . . . 15
8.3 Bad-pixel replacement . . . 16
8.4 Where the table is . . . 16
9 Corner turn and scene guard . . . 16
9.1 Why the sequence must change . . . 16
9.2 How it operates . . . 17

9.3 When the compressor is slow: fill lines . . . 17
9.4 The scene guard . . . 17
10 CCSDS 123 compression . . . 18
10.1 Lossless compression . . . 18
10.2 The idea: predict, then send the error . . . 18
10.3 The neighbourhood . . . 18
10.4 Weights that learn . . . 18
10.5 The mapper and the entropy coder . . . 18
10.6 One scene, one image . . . 19
10.7 Why BIP order makes 250 MHz possible . . . 19
10.8 Results . . . 20
10.9 Supported settings . . . 20
11 Packets and frames . . . 21
11.1 Why the data needs labels . . . 21
11.2 Space Packets (CCSDS 133.0) . . . 21
11.3 AOS transfer frames (CCSDS 732.0) . . . 22
11.4 Idle frames . . . 22
12 Channel coding . . . 23
12.1 Reed-Solomon code . . . 23
12.2 Interleaving . . . 23
12.3 Randomizer . . . 23
12.4 The sync marker and the CADU . . . 23
13 The error injector . . . 24
13.1 Why add errors on purpose . . . 24
13.2 Modes . . . 24
13.3 Results: how far the code holds . . . 24
14 The optical link . . . 25
14.1 The problems of a serial line . . . 25
14.2 The 8b/10b code . . . 25
14.3 The link transmitter . . . 25
14.4 The GTH transceiver and the SFP module . . . 26
14.5 The link receiver . . . 26
14.6 The lock state machine . . . 26
14.7 The PRBS test . . . 26
14.8 The eye scan . . . 27
15 The ground receiver . . . 27
15.1 Frame synchronization . . . 27
15.2 Derandomizer and capture . . . 28
16 The PC ground station . . . 29
16.1 What it does . . . 29
16.2 Commands . . . 29
16.3 How the data gets to the PC . . . 29
16.4 Why the decoding is on the PC . . . 29
Part C Advanced topics . . . 30
17 Registers and software . . . 30
17.1 Registers . . . 30
17.2 One source for hardware, software and documents . . . 30
17.3 The cfg_pending flag: a lesson from M4 . . . 30

17.4 The control program hpdp_run . . . 30
17.5 The host scripts and a board run . . . 31
18 Clocks, resets and clock-domain crossings . . . 32
18.1 Why there are several clocks . . . 32
18.2 The danger at a clock crossing . . . 32
18.3 Resets . . . 32
18.4 Timing closure . . . 33
19 Data streams, back-pressure and overflow . . . 33
19.1 The AXI4-Stream handshake . . . 33
19.2 Back-pressure . . . 33
19.3 Where data can be lost, and how the HPDP reports it . . . 34
20 Protection against radiation . . . 34
20.1 What radiation does to an FPGA . . . 34
20.2 Layer 1: configuration scrubbing . . . 34
20.3 Layer 2: error-correcting memory (ECC) . . . 34
20.4 Layer 3: triple modular redundancy (TMR) . . . 35
20.5 Layer 4: watchdogs . . . 35
20.6 Layer 5: flags in the data . . . 35
20.7 The fault-injection campaign . . . 35
20.8 How the campaign changed the design . . . 36
21 How the team proved that it works . . . 36
21.1 Model first . . . 36
21.2 The test levels . . . 36
21.3 The milestones . . . 37
21.4 Problems that the team found, and the lessons . . . 37
22 Implementation results . . . 38
22.1 The 1024-pixel build . . . 38
23 Results summary . . . 39
24 Status, limits and next steps . . . 39
24.1 Status . . . 39
24.2 Limits . . . 39
24.3 Possible next steps . . . 40
25 Procedures . . . 40
25.1 Prepare the board . . . 40
25.2 Run the model tests . . . 40
25.3 Run the hardware regression . . . 40
25.4 Build the FPGA configuration . . . 40
25.5 Build the program . . . 41
25.6 Do a board run . . . 41
25.7 Decode the run at the ground station . . . 41
25.8 If a fault occurs . . . 41
Appendix A Answers to the check questions . . . 42
Appendix B Numbers to remember . . . 46
Appendix C Where things are in the repository . . . 47
Appendix D Technical names and abbreviations . . . 48

Part A The basics

Part A is for a reader with no special knowledge. It tells what the HPDP is, which problems it solves and which basic ideas you must know before you read Part B.

1 About this guide

1.1 Purpose

This guide teaches you the HPDP from the start. HPDP is the short name for the Hyperspectral Payload Data Processor. The guide starts with simple ideas. Each chapter adds more detail. At the end, you will know what each part of the system does, why it is there and how the project team tested it.

1.2 How to read this guide

The guide has three parts. Read them in sequence, because each part uses the words and the ideas of the part before it.

Table 1: The three parts of this guide

Part	Level	Contents
A	Basic	The problem, the basic ideas (bits, data rate, clock, image cube), the hardware and the data chain at a glance
B	Intermediate	Each block of the data chain: what it does, how it does it, a worked example and the measured results
C	Advanced	Registers and software, clocks, data streams, radiation protection, the verification method, the results and the procedures

Each chapter ends with two boxes:

- **KEY POINTS** gives the most important facts of the chapter.
- **Check your knowledge** gives some questions. Try to answer them before you continue. Appendix A gives the answers.

Appendix B gives the important numbers on one page. Appendix C tells where each part of the project is in the repository. Appendix D gives the technical names and abbreviations.

1.3 Writing rules

This guide uses ASD-STE100 Simplified Technical English (STE). STE is a controlled language for technical documents. These are the rules that you will see:

- A sentence in a description has a maximum of 25 words.
- A sentence in a procedure has a maximum of 20 words. Each step gives one instruction.
- Instructions use the imperative form of the verb, for example “Connect the fiber”.
- Each word has only one meaning.
- Technical names, for example “Reed-Solomon code”, are used where necessary. Appendix D tells what they mean.

1.4 Warnings, cautions and notes

Table 2: Warnings, cautions and notes

Type	Meaning
WARNING	Tells you about a risk of injury to persons.
CAUTION	Tells you about a risk of damage to the equipment, to the data or to the test results.
NOTE	Gives information that helps you.

2 The HPDP in one page

2.1 What the HPDP is

An Earth observation satellite carries a camera. The camera records the ground. The satellite must then send the images to a station on the ground. Between the camera and the radio transmitter, there is an electronic unit that prepares the data. This unit is the payload data processor.

The HPDP is a working model of this unit. It does not fly. It operates on one AMD ZCU106 evaluation board on a desk. The board also contains the receiver of the ground station, so that one board can do the full test.

The HPDP does not change the image. The ground station gets back each bit of the image exactly as the satellite had it. The project calls this result **bit-exact**. Bit-exact is the pass criterion of almost all tests in this project.

2.2 The four problems that the HPDP solves

Figure 1 shows the four problems and the answer of the HPDP to each problem.

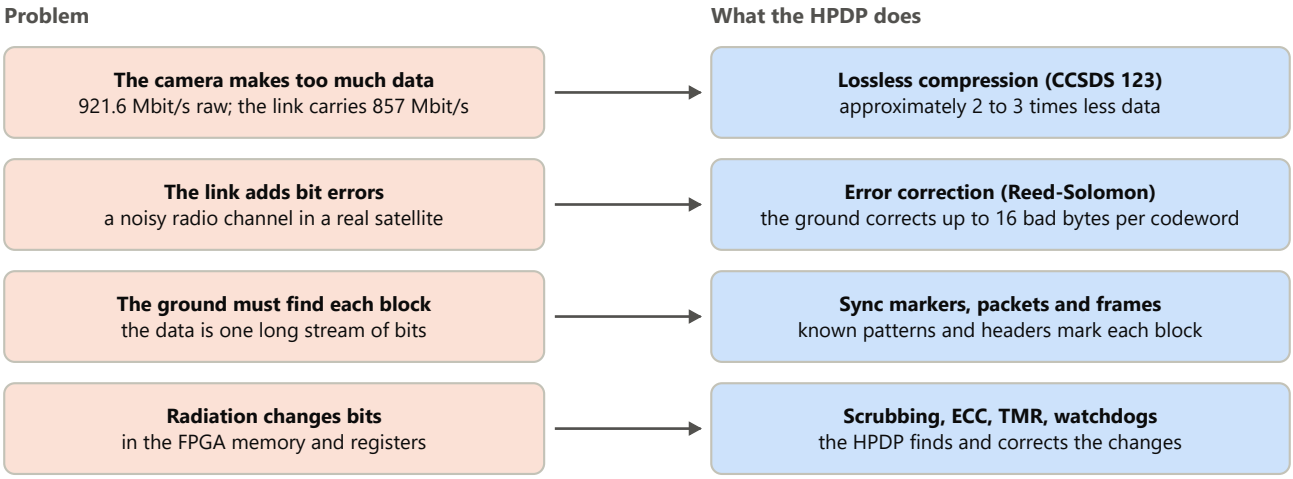


Figure 1: The four problems of a satellite data chain and the answers of the HPDP

You can compare the data chain to a shipment of books through a narrow pipe that sometimes damages the parcels:

- Compression packs the books tightly, so that more books go through the pipe.
- Error correction adds spare information, so that the receiver can repair a damaged parcel.
- Packets, frames and sync markers put labels on each parcel, so that the receiver knows where each parcel starts and which book it holds.
- Radiation protection checks the packing machine all the time, because space radiation can damage it.

2.3 What the HPDP is not

The HPDP shows the architecture, the standards and the test method of a real payload processor. Some things are not in the project:

- The parts are commercial parts, not radiation-tolerant parts.
- There is no radio transmitter. A 1-metre optical fiber is used in place of the radio link.
- There is no real camera. The board reads recorded images from memory and sends them at the speed of a real camera.

KEY POINTS

- The HPDP is a working model of the data processor of a hyperspectral satellite.
- It compresses the data, adds error correction and labels, sends the data over a fiber and gets the image back at a ground receiver.
- The pass criterion is bit-exact: the ground image is identical to the original, bit for bit.

Check your knowledge (the answers are in Appendix A)

1. Why must a satellite compress its image data?
2. What does “bit-exact” mean?
3. Which part of a real satellite does the optical fiber replace?

3 Basic ideas that you must know

3.1 Bits, bytes and samples

A **bit** has one of two values: 0 or 1. A **byte** is a group of 8 bits. A byte has 256 possible values (0 to 255).

A camera measures the light at each point. Each measurement is a number. In this project, one such number is a **sample**. The HPDP reference camera uses 12 bits for each sample, so a sample has a value from 0 to 4095. The hardware keeps each sample in a 16-bit container, because 16 bits are easier to move.

3.2 Data rate

The data rate tells how many bits go through a point in one second. The unit is bit/s. 1 Mbit/s is one million bits each second. 1 Gbit/s is one thousand million bits each second.

Worked example: the raw data rate of the HPDP camera

The camera records 1000 lines each second.

Each line has 512 pixels, and each pixel has 150 colour bands.

Samples each second = 1000 × 512 × 150 = 76,800,000 = 76.8 Msamples/s.

Bits each second = 76.8 Msamples/s × 12 bits = 921.6 Mbit/s.

The link can carry approximately 857 Mbit/s of user data. Thus the raw data does not fit. Chapter 6 shows the full calculation.

3.3 Clock and clock cycle

Digital logic does its work in steps. A **clock** is a signal that gives the time of each step. One step is one **clock cycle**. The clock frequency tells how many steps occur in one second.

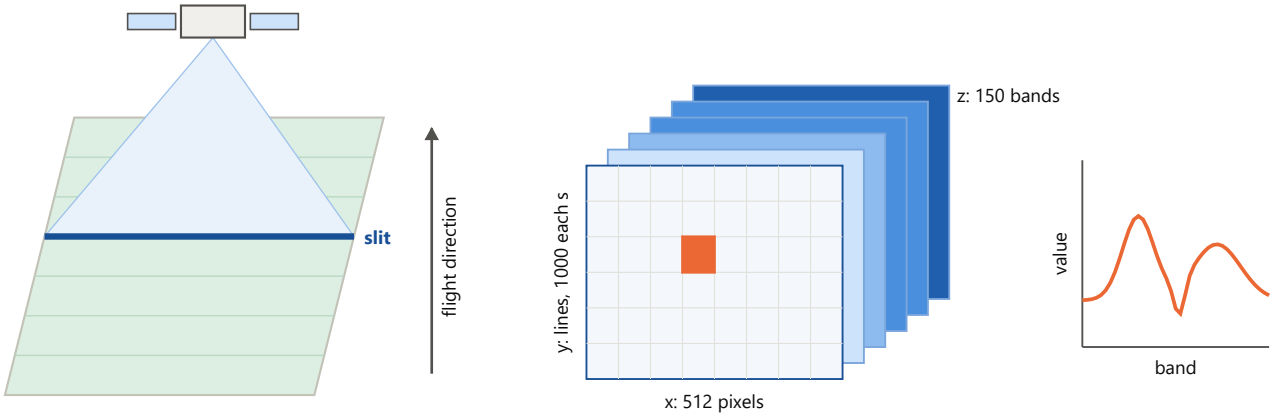
The main HPDP clock (clk_sys) has a frequency of 250 MHz. Thus one clock cycle is 4 nanoseconds. Each block of the data chain accepts one sample (or one byte) in each clock cycle. At 250 MHz, the compressor can thus accept 250 Msamples/s. This is more than three times the 76.8 Msamples/s of the camera.

3.4 Hyperspectral images

A normal colour camera records three bands of colour: red, green and blue. A **hyperspectral** camera records many narrow bands. The HPDP reference camera records 150 bands. The set of values of one pixel in all bands is the **spectrum** of that pixel. Different materials (water, plants, rock, metal) have different spectra. Thus a scientist can identify materials from space.

A hyperspectral image has three dimensions. Figure 2 shows them:

- x: the position across the track of the satellite (512 pixels).
- y: the position along the track. Each new line is a new y (1000 lines each second).
- z: the spectral band (150 bands).



(a) Pushbroom scan *(b) Image cube: one cell is one sample* *(c) One spectrum*

Figure 2: Pushbroom imaging, the image cube and the spectrum of one pixel

One **scene** is a part of the cube with a fixed number of lines. The HPDP uses 256 lines for each scene. One scene thus has 512 × 256 × 150 = 19,660,800 samples. The HPDP compresses each scene independently. An error that destroys one scene thus cannot damage the next scene.

3.5 Pushbroom imaging

The HPDP camera is a **pushbroom** camera. It does not take a full picture at one time. It looks through a narrow slit and records one line of the ground. The satellite moves forward, so the next line is a new strip of ground. A broom that you push along the floor makes the same pattern. That is the origin of the name.

A pushbroom camera cannot stop. The ground moves under it at a constant speed. If the electronics cannot accept a line at the correct time, that line is lost. This fact is important for the design of the HPDP. Chapter 7 tells how the HPDP handles it.

3.6 Streams and blocks

The data moves through the HPDP as a **stream**: a long sequence of samples or bytes, one after the other. Some blocks cut the stream into parts of fixed length, for example packets and frames. Other blocks change each value but keep the stream length.

3.7 Standards and CCSDS

A satellite and its ground station must use the same data formats. The Consultative Committee for Space Data Systems (CCSDS) writes these formats as standards. Space agencies all over the world use them. The HPDP follows four CCSDS standards:

Table 3: The CCSDS standards that the HPDP uses

Standard	Subject	Used in
CCSDS 123.0-B-2	Lossless compression of multispectral and hyperspectral images	Chapter 10
CCSDS 133.0-B	Space Packet Protocol	Chapter 11
CCSDS 732.0-B	AOS Space Data Link Protocol (transfer frames)	Chapter 11
CCSDS 131.0-B	Synchronization and channel coding (Reed-Solomon, randomizer, sync marker)	Chapter 12

KEY POINTS

- A sample is one measured number. The HPDP camera uses 12-bit samples.
- The raw data rate is 1000 lines × 512 pixels × 150 bands × 12 bits = 921.6 Mbit/s.
- At 250 MHz, one clock cycle is 4 ns. Each block moves one sample or byte in each cycle.
- A hyperspectral image is a cube: x (pixels), y (lines), z (bands). A scene is 256 lines.
- A pushbroom camera cannot stop, so the electronics must keep pace with it.

Check your knowledge (the answers are in Appendix A)

1. How many values can a 12-bit sample have?
2. The clock is 250 MHz. How long is one clock cycle?
3. How many samples does one HPDP scene have?
4. Why is it a problem when the electronics cannot accept a line from a pushbroom camera?

4 The hardware

4.1 What an FPGA is

An FPGA (field-programmable gate array) is a chip that contains a large quantity of small logic parts. You do not program it like a processor. You describe a digital circuit in a hardware description language, and a tool connects the parts of the FPGA to make that circuit. The HPDP team wrote the circuit in SystemVerilog. The tool is AMD Vivado.

The circuit in an FPGA does many things at the same time. Each block of the data chain is a separate circuit, and all blocks operate in each clock cycle. This is why an FPGA can move hundreds of millions of samples each second.

Table 4: The main resources of the FPGA

Resource	What it is	Quantity in the XCZU7EV
LUT (lookup table)	A small logic function of up to 6 inputs	230,400
Flip-flop (FF)	A 1-bit memory that changes at a clock edge	460,800
Block RAM (BRAM)	A 36-kilobit memory	312
UltraRAM (URAM)	A 288-kilobit memory, the largest memory in the chip	96
DSP slice	A fast multiplier and adder	1,728
GTH transceiver	A serial transmitter and receiver for fast links	used: 2

4.2 The Zynq device: PS and PL

The ZCU106 board has an AMD Zynq UltraScale+ device, type XCZU7EV. This device has two parts in one chip:

- The **processing system (PS)**. It has four Arm Cortex-A53 processors and a DDR4 memory controller. One processor runs the HPDP control program, hpdp_run.
- The **programmable logic (PL)**. This is the FPGA part. It contains the full data chain.

The PS and the PL connect through AXI buses. AXI is a standard set of on-chip connections. The PS uses AXI to write the settings of the data chain. The DMA engines use AXI to move data between the DDR4 memory and the PL.

4.3 The ZCU106 board

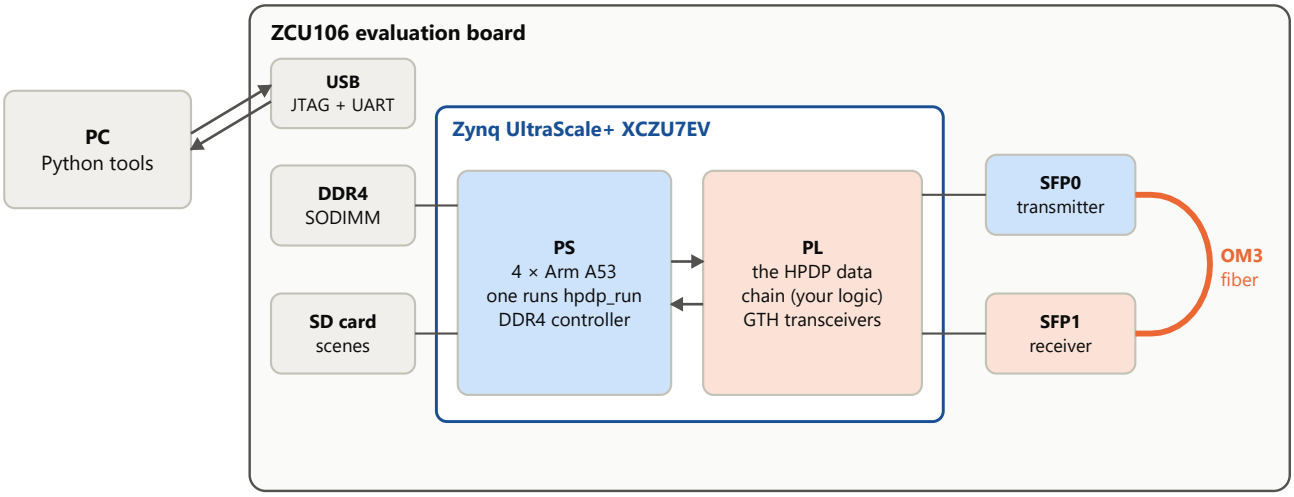


Figure 3: The parts of the ZCU106 board that the HPDP uses

Table 5: Board parts and their use

Part	Use in the HPDP
DDR4 memory (SODIMM)	Holds the raw scenes, the calibration tables and the received data
SD card	Holds raw scenes and tables for the UART console (optional)
USB: JTAG	Loads the FPGA configuration, the program and the test data from the PC; reads the results back
USB: UART	Text console of the program, 115200 bit/s
SFP0 cage	The satellite transmitter: an 850 nm optical module
SFP1 cage	The ground receiver: a second optical module
Si570 oscillator	Gives the 156.25 MHz reference clock of the transceivers

4.4 SFP modules and the optical fiber

An SFP module is a small plug-in unit that changes electrical signals into light and light into electrical signals. The HPDP uses two 1000BASE-SX modules. They send infrared light with a wavelength of 850 nm.

A 1-metre OM3 multimode fiber connects SFP0 to SFP1. Thus the light goes from the satellite side to the ground side of the same board. The data goes in one direction only, like a real satellite downlink.

> **NOTE**
> The 1000BASE-SX modules are Class 1 laser products, which are safe in normal use. The 850 nm light is not visible. As a good practice, do not look into an open SFP port or into the end of a fiber.

> **CAUTION**
> Do not bend the fiber tightly. Put the dust caps on the modules and on the fiber ends when you disconnect them. Dirt on a fiber end causes bit errors.

KEY POINTS

- An FPGA is a chip of logic parts that you connect to make your own circuit.
- The Zynq device has a PS (four Arm processors) and a PL (the FPGA). The data chain is in the PL; the control program runs in the PS.
- The PC talks to the board through USB (JTAG and UART).
- Two SFP modules and a fiber make the optical link on the same board.

Check your knowledge (the answers are in Appendix A)

1. What is the difference between the PS and the PL?
2. Which memory type is the largest on-chip memory, and how many does the XCZU7EV have?
3. What does an SFP module do?

5 The data chain at a glance

5.1 The blocks

Figure 4 shows all blocks of the data chain. Read the first row from left to right, the second row from right to left and the third row from left to right.

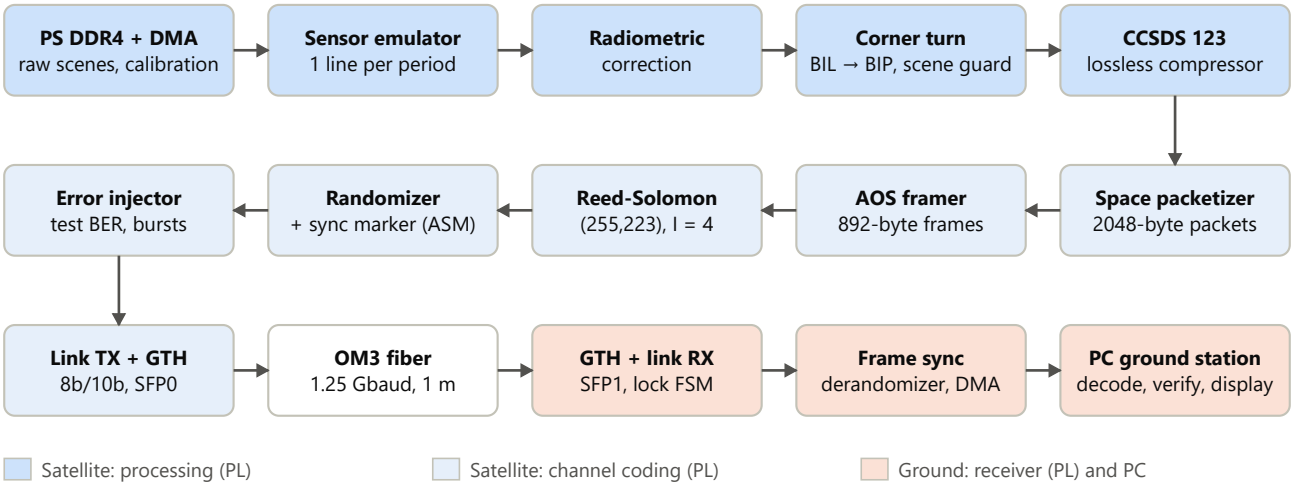


Figure 4: The HPDP data chain

5.2 The journey of one sample

Follow one sample through the full chain:

1. The PC loads a raw scene into the DDR4 memory of the board.
2. A DMA engine reads the scene. The sensor emulator sends one line at the time of a real camera.
3. The radiometric correction removes the errors of the detector from the sample.
4. The corner turn changes the sequence of the samples, so that all bands of one pixel come together.
5. The CCSDS 123 compressor predicts the sample from its neighbours. It sends only the small difference, with a short code.
6. The space packetizer puts the compressed bits into packets with a header.
7. The AOS framer puts the packets into frames of fixed length.
8. The Reed-Solomon encoder adds parity bytes to each frame. The randomizer mixes the bits. A sync marker goes in front. The result is a CADU of 1024 bytes.

9 The error injector can add bit errors on purpose, to test the error correction.

10 The link transmitter changes each byte into a 10-bit symbol. The GTH transceiver sends the symbols as light through the fiber at 1.25 Gbaud.

11 The ground receiver finds the sync markers, removes the randomization and writes the data into the DDR4 memory.

12 The PC reads the data. The ground station program corrects the errors, unpacks the frames and packets and decompresses the image.

13 The ground station compares the image with the original. The result must be bit-exact.

5.3 The satellite side and the ground side

In a real mission, the satellite and the ground station are hundreds of kilometres apart. In the HPDP, both are on one board. The blocks from the sensor emulator to the link transmitter are the satellite side. The blocks from the link receiver to the ground station program are the ground side. The ground side does not use any information from the satellite side, except the data that comes through the fiber. Thus the test is honest.

NOTE

The ground station program on the PC does the Reed-Solomon decoding and the decompression in Python. The FPGA ground receiver only finds the frames and stores them. Chapter 16 tells why.

KEY POINTS

• The chain has three groups. The satellite processing does correction, corner turn and compression. Channel coding adds packets, frames, Reed-Solomon and sync markers. The ground has the receiver and the PC.
• Both sides are on one board, and only the fiber connects them.

Check your knowledge (the answers are in Appendix A)

1. Which block puts the compressed bits into packets?
2. How long is one CADU?
3. Which program does the decompression?

Part B How each block works

Part B goes through the data chain block by block, in the sequence of the data. For each block, it tells what the block does, why it is necessary, how it operates and what the tests measured.

6 The data budget

6.1 Why the budget is the start of the design

Before the team designed a block, they calculated how much data the camera makes and how much the link can carry. This calculation is the data budget. It shows that compression is necessary, and it tells how good the compression must be.

6.2 The calculation

Table 6: The data budget of the reference mission

Quantity	Value	How to calculate it
Raw data (12-bit samples)	921.6 Mbit/s	76.8 Msamples/s × 12 bits
Link rate	1.25 Gbaud	symbols each second on the fiber
Link payload after 8b/10b	1000 Mbit/s	1250 × 8 / 10 (Chapter 14)
CADU efficiency	86.3 %	884-byte packet zone / 1024-byte CADU (Chapters 11, 12)
Space packet efficiency	99.3 %	2048 data bytes / 2062-byte packet
User data capacity	**≈ 857 Mbit/s**	1000 × 0.863 × 0.993
Does raw data fit?	**No**	921.6 is more than 857
Compressed at 2:1	460.8 Mbit/s	54 % of the capacity
Compressed at 3:1	307.2 Mbit/s	36 % of the capacity

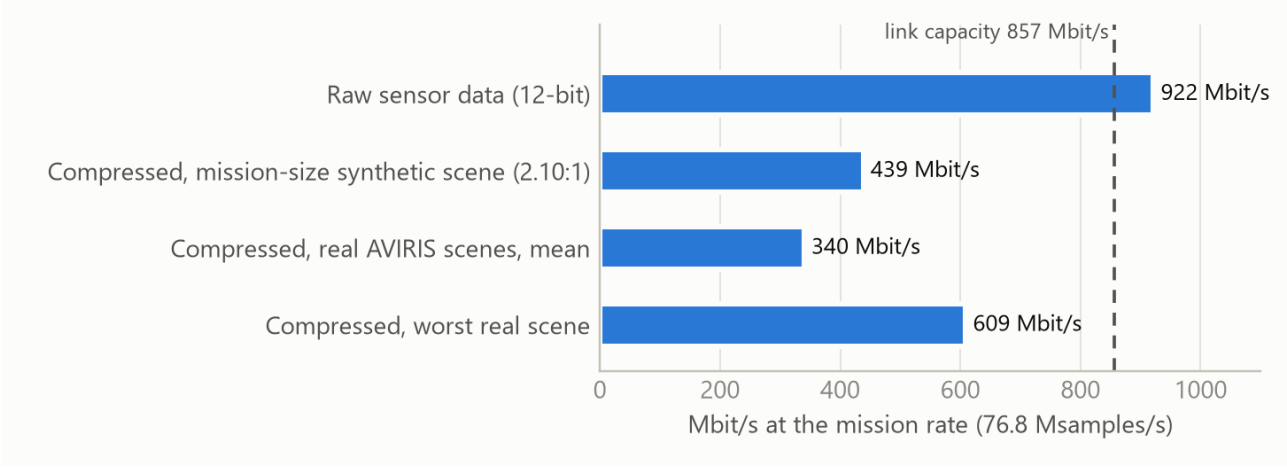


Figure 5: Raw and compressed data rates against the link capacity

The figure shows the measured results. The mission-size test scene needs 439 Mbit/s. The average real AVIRIS scene needs 340 Mbit/s. The worst real scene of the 63 that the team tested needs 609 Mbit/s, which is 71 % of the capacity. All of them fit.

6.3 The proof that raw data does not fit

The HPDP has a RAW mode. In this mode, the compressor sends the samples without compression. Requirement SYS-09 says that RAW mode must show lost lines at the mission rate. The board test confirmed it: 99.1 % of the lines were dropped, and the HPDP counted each dropped line. In COMPRESS mode at the same rate, no line was dropped in 10 minutes.

KEY POINTS

- The link carries approximately 857 Mbit/s of user data, but the camera makes 921.6 Mbit/s.
- Compression of approximately 2:1 or better makes the data fit with a good margin.
- RAW mode shows the problem on the board: almost all lines are lost.

Check your knowledge (the answers are in Appendix A)

1. Why is the user data capacity (857 Mbit/s) less than the link payload (1000 Mbit/s)?
2. What percentage of the link does a scene with 2:1 compression use?
3. Which mode shows that raw data does not fit, and what did the board test show?

7 The sensor emulator

7.1 What it does

The HPDP has no real camera. The sensor emulator takes the place of the camera. It reads a recorded raw scene from the DDR4 memory. It sends the scene line by line, at the time and in the sequence of a real pushbroom detector.

7.2 How it operates

1. A DMA engine (AXI DMA, MM2S direction) reads the scene from DDR4. The data goes into a 32 KB prefetch buffer, so that the next line is ready before it starts.
2. A counter on clk_sensor (100 MHz) starts a new line every LINE_PERIOD clock cycles.
3. At the start of a line, the emulator sends all samples of the line, one in each clock cycle, with no gap.
4. It sends them in BIL order: band 0 of all pixels, then band 1 of all pixels, and so on.
5. It marks the first sample of each line and of each scene, so that later blocks know where the lines and scenes start.

Worked example: the line period for the mission rate

The mission rate is 1000 lines each second, thus one line each millisecond.

clk_sensor is 100 MHz, thus 1 ms = 100,000 clock cycles. Set LINE_PERIOD to 100,000.

One line has 512 × 150 = 76,800 samples. The emulator needs 76,801 cycles to send it (0.768 ms).

For the rest of the millisecond, the emulator waits for the next line start.

7.3 It never waits

A real detector cannot wait for the electronics. Thus the emulator never waits for the next block. If the next block cannot accept a sample, the emulator drops the sample. It marks the line and counts the event. The HPDP never loses data without a record of the loss. Requirement SYS-05 calls this rule “never silent”.

If the DMA does not deliver the data in time, the emulator sends zeros and counts an underrun. An underrun is a fault of the test setup, not of the design.

7.4 Other functions

- Line and scene counters, and a 24-bit line timestamp. The timestamp at the start of each scene goes into the packet headers (Chapter 11).
- A built-in ramp and a pseudo-random (LFSR) pattern generator. They let you test the chain without DMA.

7.5 Measured results

In the M4 tests, the emulator sent up to 99.8 Msamples/s, its maximum rate. The compressor can accept 250 Msamples/s, thus the emulator, not the compressor, set the speed limit of these tests.

> **KEY POINTS**
> • The sensor emulator replays recorded scenes with the timing of a real pushbroom camera.
> • It sends each line in BIL order, one sample each clock cycle.
> • It never waits. Lost samples and late data are counted, never silent.

Check your knowledge (the answers are in Appendix A)

1. Which value of LINE_PERIOD gives 1000 lines each second?
2. What does the emulator do when the next block cannot accept a sample?
3. What is an underrun?

8 Radiometric correction

8.1 Why a detector needs correction

A detector has many small light sensors, one for each pixel and band. These sensors are not all equal:

- **Dark offset.** A sensor gives a small value even when it receives no light.
- **Gain.** Two sensors that receive the same light can give different values, because their sensitivity is different.
- **Bad pixels.** Some sensors are dead (always low) or hot (always high).

If you do not correct these errors, the image has stripes, and the compressor works less well. The correction uses a calibration table. It has three values for each band and pixel: dark (16 bits), gain (16 bits) and a bad-pixel flag (1 bit).

8.2 The formula

For a raw sample x, the block calculates the corrected value c:

```
c = clamp( ((x − dark) × gain + 2^13) >> 14 ,  0 ,  2^D − 1 )
```

- The gain is a fixed-point number with 14 fraction bits. 0x4000 (16384) means a gain of 1.0.
- Adding 2^13 (8192) before the shift by 14 bits rounds the result to the nearest integer.
- Clamp keeps the result between 0 and the maximum value (4095 for D = 12). The block counts each clamp.
- One DSP slice does the multiplication and the addition in one step.

Worked example: one corrected sample

Raw sample x = 1000, dark = 100, gain = 0x4800 = 18432 (this is 1.125).
(1000 − 100) × 18432 = 16,588,800.
16,588,800 + 8192 = 16,596,992.
16,596,992 >> 14 = 1013. Check: 900 × 1.125 = 1012.5, and this rounds up to 1013.
1013 is between 0 and 4095, thus c = 1013.

8.3 Bad-pixel replacement

When the flag of a pixel shows “bad”, the block replaces the value. It uses the neighbours in the same band and the same line:

Table 7: Bad-pixel replacement rules

Condition	New value
The pixel is good	its own corrected value c[x]
Both neighbours are good	the average: (c[x−1] + c[x+1] + 1) >> 1
Only the left neighbour is good	c[x−1]
Only the right neighbour is good	c[x+1]
No neighbour is good	0, and the block counts the event

Worked example: a bad pixel

The left neighbour is 1010 and the right neighbour is 1020. Both are good.
New value = (1010 + 1020 + 1) >> 1 = 2031 >> 1 = 1015.

The data comes in BIL order. Thus the left and right neighbours in a band are next to the pixel in time. The block needs only a window of three samples. This is a good reason for the correction to come before the corner turn.

8.4 Where the table is

The calibration table is in UltraRAM. A DMA transfer loads it before a run. The table for 512 pixels and 256 bands uses 32 UltraRAMs, with error correction (ECC, Chapter 20). For the 1024-pixel build, the table packs two entries into each memory word, so that it fits (Chapter 22).

NOTE

The public hyperspectral images that the team used for tests are already corrected. To test this block, the team applied synthetic dark, gain and bad-pixel maps in reverse. This made “raw” data with a known correct answer.

KEY POINTS

- The correction removes dark offset and gain differences and replaces bad pixels.
- c = clamp(((x − dark) × gain + 2^13) >> 14, 0, 2^D − 1); 0x4000 means a gain of 1.0.
- A bad pixel gets the average of its good neighbours in the same band and line.

Check your knowledge (the answers are in Appendix A)

1. A sensor gives a value when there is no light. What is the name of this error?
2. Which gain value in the table means “no change”?
3. x = 500, dark = 100, gain = 0x2000. What is c?
4. A bad pixel has a bad left neighbour and a good right neighbour (value 700). What is the new value?

9 Corner turn and scene guard

9.1 Why the sequence must change

The sensor sends samples in BIL order (band-interleaved by line): all pixels of band 0, then all pixels of band 1. The compressor needs BIP order (band-interleaved by pixel): all bands of pixel 0, then all bands of pixel 1. Section 10.7 tells why. The corner turn makes this change.

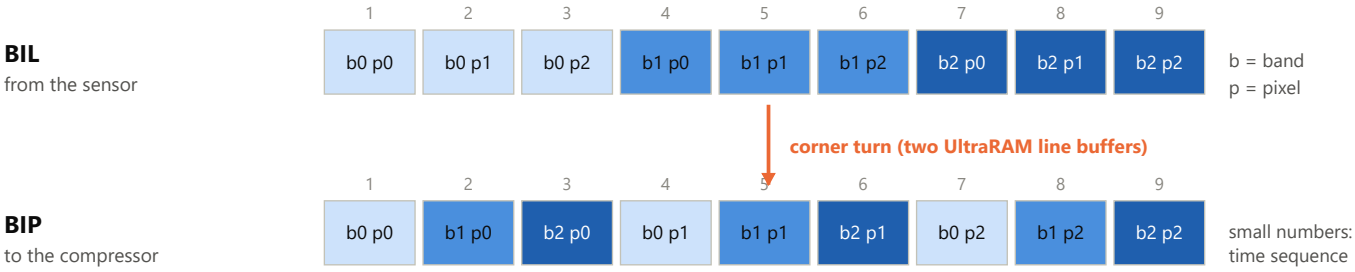


Figure 6: BIL and BIP order for 3 pixels and 3 bands

9.2 How it operates

The corner turn has two line buffers in UltraRAM. Each buffer holds one full line (512 pixels × 150 bands). It writes line n into one buffer and at the same time reads line n−1 from the other buffer in the new sequence. At the end of each line, the two buffers change roles. This method is “ping-pong” buffering. Both sides move one sample in each clock cycle. The delay is approximately one line.

9.3 When the compressor is slow: fill lines

The compressor can stop the corner turn for a short time (back-pressure). Sometimes both buffers are still full when the next line starts. Then the corner turn drops the new line, counts it and sends a line of zeros in its place. This line of zeros is a **fill line**.

Why send zeros, and not just drop the line? Each scene must have exactly 256 lines. If a line is missing, all the following lines move up by one, and the compressor loses the boundaries of the scenes. A fill line keeps the geometry correct. The packet header of the scene then carries the flag “lines dropped”.

9.4 The scene guard

The team found a second problem in the M5 tests, under a very high overload. Sometimes even a fill line could not go out. Then a scene arrived short, the compressor did not end it, and the next scenes merged into it. Only 13 of 234 scenes were complete.

The scene guard solves this problem. It sits between the corner turn and the compressor. Sometimes a new scene starts before the current scene is complete. Then the scene guard holds the input and adds zeros until the current scene is complete. The compressor then ends the scene normally. After this change, all 218 scenes that reached the compressor were delivered.

KEY POINTS

- The corner turn changes BIL order into BIP order with two ping-pong line buffers.
- If the compressor is too slow, a dropped line becomes a counted fill line of zeros.
- The scene guard makes sure that each scene that the compressor sees has its full size.

Check your knowledge (the answers are in Appendix A)

1. In BIP order, which sample comes after pixel 0, band 0?
2. Why does the corner turn need two line buffers?
3. Why does the HPDP send a fill line in place of a dropped line?

10 CCSDS 123 compression

10.1 Lossless compression

Compression makes the data smaller. **Lossless** compression lets the receiver rebuild the data exactly. Scientists need the exact values, so the HPDP uses lossless compression. The method is the CCSDS 123.0-B-2 standard. It is made for multispectral and hyperspectral images, and it is simple enough for space hardware.

10.2 The idea: predict, then send the error

Neighbouring samples are similar. A pixel is similar to the pixels next to it. Much more important, the value of a pixel in one band is very similar to its value in the band before. The compressor uses this fact:

1. It predicts the value of the next sample from samples that it has already sent.
2. It calculates the residual: actual value minus predicted value.
3. It sends only the residual. A good prediction gives a small residual, and a small number needs fewer bits.

The decoder on the ground makes the same prediction from the same samples, because it has already decoded them. It adds the residual and gets the exact value back. Thus no information is lost.

10.3 The neighbourhood

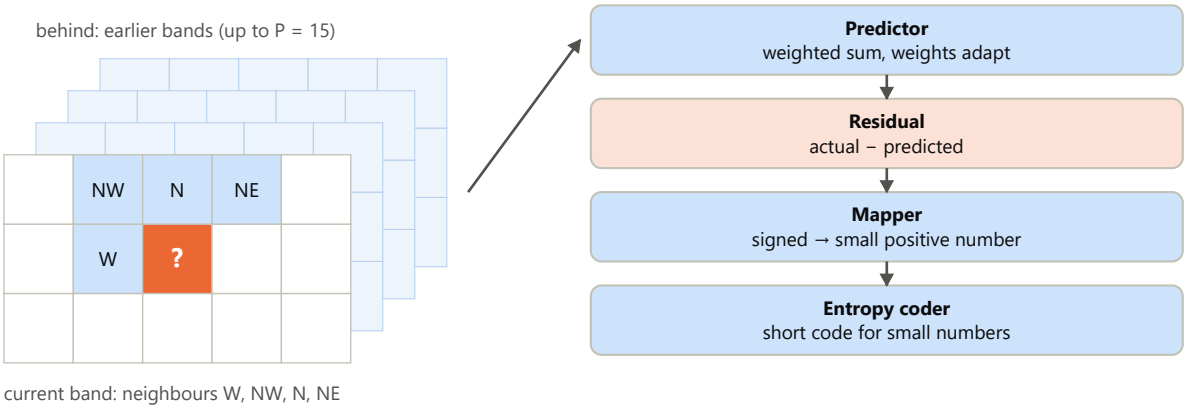


Figure 7: The samples that the predictor uses, and the steps after the prediction

- In the current band, the predictor uses the neighbours W (west), NW, N (north) and NE. They make the **local sum**.
- It also uses up to P = 15 earlier bands of the same pixel. From them, it takes the **local differences**: how each band differed from its own local sum.
- The prediction is the local sum plus a weighted sum of the local differences.

10.4 Weights that learn

Each band has its own set of weights. After each sample, the predictor compares its prediction with the actual value. It then changes the weights a small step in the direction that makes the error smaller. The weights thus adapt to the image while the compression continues. The parameters of this adaptation are the weight resolution (Ω = 13 by default) and the update scaling values (ν_{min}, ν_{max}, t_{inc}).

10.5 The mapper and the entropy coder

A residual can be positive or negative. The **mapper** changes it into a number that is zero or more. Small residuals become small numbers.

The **entropy coder** then writes each number with a variable number of bits. The HPDP uses the sample-adaptive coder of the standard. For each band, it keeps a running average of the recent numbers. From this average, it selects a code parameter k:

- The code is: (number >> k) zeros, then a one, then the k lowest bits of the number.
- A band with small residuals gets a small k, thus short codes.
- A band with large residuals gets a larger k, so that the codes do not become too long. The limit U_{max} (18 by default) stops very long codes.

Worked example: one sample

The neighbours and the earlier bands give a predicted value of 1843. The actual value is 1846.

Residual = 1846 − 1843 = +3. The mapper changes it into a small number, for example 6.

With k = 2: 6 >> 2 = 1, thus one zero, then a one, then the 2 low bits of 6 (binary 10).

Code = 0 1 10, which is 4 bits in place of 12 bits.

A bit packer joins all codes into bytes. In front of each scene, the compressor writes a header. The header gives the image size and the settings, so that the decoder can start.

10.6 One scene, one image

The compressor starts each scene with new weights, new coder statistics and a full header. Thus each scene can be decoded alone. If the link destroys a part of one scene, the next scene is not affected (requirement SYS-06).

10.7 Why BIP order makes 250 MHz possible

This is the most important design idea of the compressor. The weights and the coder statistics of a band change after each sample of that band. In BIP order, the next sample of the same band comes Nz clock cycles later (150 cycles for the mission). Thus the circuit has Nz cycles to finish the update.

In lossless mode, the predictor uses the original samples, not values that it calculated itself. Thus there is no loop from one sample to the next. The only loops are the per-band weights and statistics, and they have Nz cycles of time. The team could thus make a deep pipeline of approximately 25 stages, plus the packer:

Table 8: The pipeline of the compressor

Stages	Function
S1 – S3	Neighbourhood: read the line buffer (UltraRAM) and the delay line
S4 – S5	Local sums and local differences
P1 – P15	Predictor and weight update
C0 – C4	Mapper and sample-adaptive coder
Packer	Codes into bytes, through a 32-entry codeword buffer

The pipeline depth sets the minimum number of bands. A band must not come back before its previous update is complete. Thus the HPDP needs at least 16 bands (NZ_MIN = 16). When the next block applies back-pressure, one enable signal stops the full pipeline. No data is lost.

The compressor uses 13.6 k LUTs, 6.2 k flip-flops, 19 DSP slices, 5.5 block RAMs and 8 UltraRAMs, and it meets 250 MHz. The team packed four samples into each UltraRAM word of the line buffer. This change reduced the line buffer from 32 to 8 UltraRAMs and also made the timing better.

10.8 Results

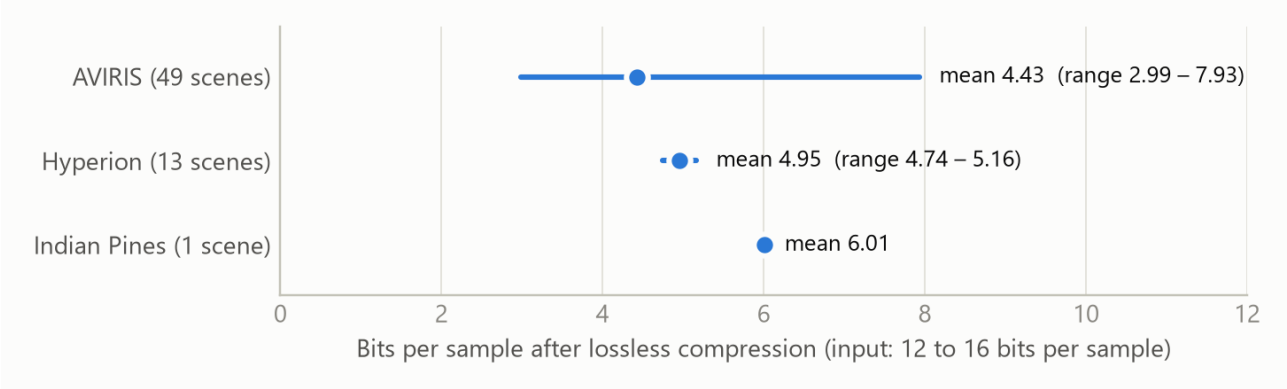


Figure 8: Compression of 63 real hyperspectral scenes

Table 9: Measured compression results

Data	Scenes	Bits per sample, mean (range)	Ratio	Link use, mean (worst)
AVIRIS (airborne), 512 × 512 × 224	49	4.43 (2.99 – 7.93)	3.12:1	40 % (71 %)
Hyperion (EO-1 satellite), 256 × 1024 × 242	13	4.95 (4.74 – 5.16)	2.86:1	44 % (46 %)
Indian Pines (AVIRIS), 145 × 145 × 220	1	6.01	2.33:1	54 %
Synthetic ramp pattern, D = 10	–	1.20	8.30:1	–
Random (LFSR) pattern, D = 14	–	14.50	0.97:1	–

- All 63 real scenes were lossless: the decoder rebuilt each one exactly.
- CCSDS 123 needs 1.6 to 2.0 times fewer bits than ZIP (deflate) on the same data.
- Without the earlier bands (P = 0), the scenes need 12 % to 67 % more bits. The spectral prediction is the most important part.
- Random data cannot be compressed. The LFSR pattern became 3 % larger, because the coder adds a small overhead. This is normal for all lossless methods.

10.9 Supported settings

All settings come from registers. You do not have to build the FPGA again to change them. The team tested all 864 combinations of the main settings (Chapter 21).

Table 10: Compressor settings

Setting	Range in the HPDP	Default
Sample depth D	2 to 16 bits	12
Earlier bands for prediction P	0 to 15	3
Prediction mode	full or reduced	full
Local sum type	wide or narrow, neighbour- or column-oriented	wide, neighbour
Weight resolution Ω	4 to 19	13
Code length limit U_{max}	8 to 32	18
Image size	Nx up to 512 (1024 build), Nz 16 to 256	512 × 256 × 150

KEY POINTS

• The compressor predicts each sample from neighbours and earlier bands and sends only the residual, with a short adaptive code.
• The ground makes the same prediction, thus the result is lossless.
• BIP order gives each band Nz cycles to update its state. Thus the pipeline can be deep and fast: 1 sample each clock at 250 MHz.
• Real scenes compress 2.3:1 to 3.1:1 on average; all 63 were lossless.

Check your knowledge (the answers are in Appendix A)

1. Why can the decoder make the same prediction as the encoder?
2. Which neighbours does the predictor use in the current band?
3. Why does BIP order help the hardware to run at 250 MHz?
4. Write the code for the number 9 with k = 2.
5. What happens when you set P = 0?

11 Packets and frames

11.1 Why the data needs labels

The compressed scene is a long sequence of bits. The ground station must know which scene a part belongs to, in which sequence the parts come and if a part is missing. Two layers of labels give this information: Space Packets and AOS transfer frames.

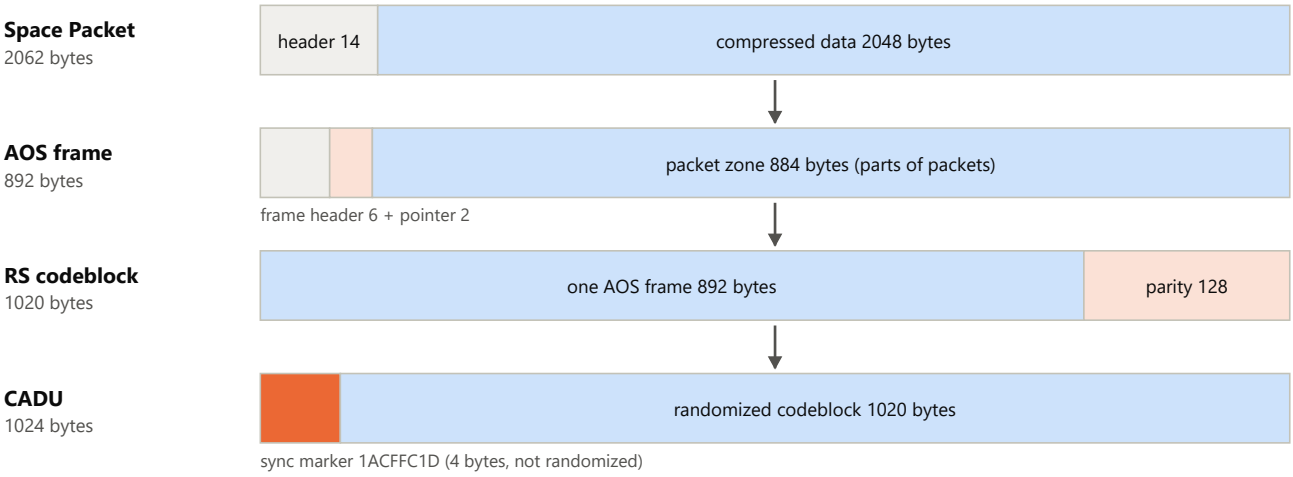


Figure 9: From packets to CADUs: each layer goes inside the next layer (not to scale)

11.2 Space Packets (CCSDS 133.0)

The space packetizer cuts the compressed scene into packets of 2048 data bytes. The last packet of a scene can be shorter. Each packet has a 6-byte primary header and an 8-byte secondary header:

Table 11: Space Packet header (primary and secondary)

Field	Size	Contents
Version, type, flags	5 bits	telemetry packet with a secondary header
APID	11 bits	the source of the data (0x100 by default)
Sequence flags	2 bits	first packet of a scene, middle, last, or the full scene in one packet
Sequence count	14 bits	increases by one for each packet

Field	Size	Contents
Data length	16 bits	length of the packet data, minus 1
Scene ID	16 bits	increases by one for each scene
Packet index	16 bits	0, 1, 2, ... inside the scene
Scene timestamp	24 bits	line timestamp at the start of the scene (Chapter 7)
Flags	8 bits	uncorrectable internal error, lines dropped, RAW mode

11.3 AOS transfer frames (CCSDS 732.0)

The AOS framer puts the packets into frames of a fixed length: 892 bytes. Why 892? It is 223 × 4. One frame then fills exactly one Reed-Solomon codeblock (Chapter 12). Each frame has:

- A 6-byte frame header: version, spacecraft ID, virtual channel ID (VCID) and a frame counter for each virtual channel.
- A 2-byte M_PDU header with the **first header pointer** (FHP).
- A packet zone of 884 bytes.

A packet does not have to fit in one frame. A long packet continues in the next frame. The first header pointer tells where the first new packet starts in the packet zone. If a frame is lost, the ground station uses the pointer of the next frame to find the next complete packet. If no packet starts in a frame, the pointer is 0x7FF.

Worked example: the first header pointer

A frame starts with the last 190 bytes of a packet that began in the previous frame.

The next packet header thus starts at byte 190 of the packet zone.

The FHP of this frame is 190.

11.4 Idle frames

A real downlink must never stop, because the receiver must stay synchronized. When no science data is ready, the framer sends an idle frame on virtual channel 63. The ground station removes the idle frames.

The ground station identifies idle frames by the bit distance of the VCID field to 63, not by an exact match. At a bit error rate of 1 in 1000, an exact match would let approximately 0.6 % of the idle frames through as false data.

The framer can also limit the rate of science frames (a register). This lets you make a slower link for tests, with the same hardware.

KEY POINTS

- Space Packets (2048 data bytes and a 14-byte header) label each part of a scene.
- AOS frames have a fixed 892 bytes, so that one frame fills one Reed-Solomon codeblock.
- The first header pointer lets the ground find packet starts, also after a lost frame.
- Idle frames (VCID 63) keep the link full when there is no science data.

Check your knowledge (the answers are in Appendix A)

1. Why is the AOS frame 892 bytes long?
2. What does the first header pointer value 0x7FF mean?
3. Why does the framer send idle frames?

12 Channel coding

Channel coding prepares each frame for a noisy channel. It has three steps: Reed-Solomon coding, randomization and the sync marker (CCSDS 131.0).

12.1 Reed-Solomon code

A Reed-Solomon (RS) code adds parity bytes to the data. The HPDP uses RS(255,223): each codeword has 223 data bytes and 32 parity bytes, total 255 bytes. The decoder on the ground can correct up to 16 bad bytes in each codeword, at any positions. If more than 16 bytes are bad, the decoder usually reports “uncorrectable”.

RS works with bytes, not bits. A byte with 1 bad bit and a byte with 8 bad bits are both one error. This makes RS strong against short groups of bad bits.

12.2 Interleaving

The HPDP uses interleaving depth I = 4. It divides each frame of 892 bytes into four codewords. Byte 1 goes to codeword 1, byte 2 to codeword 2, byte 3 to codeword 3 and byte 4 to codeword 4. Byte 5 goes to codeword 1 again, and so on.

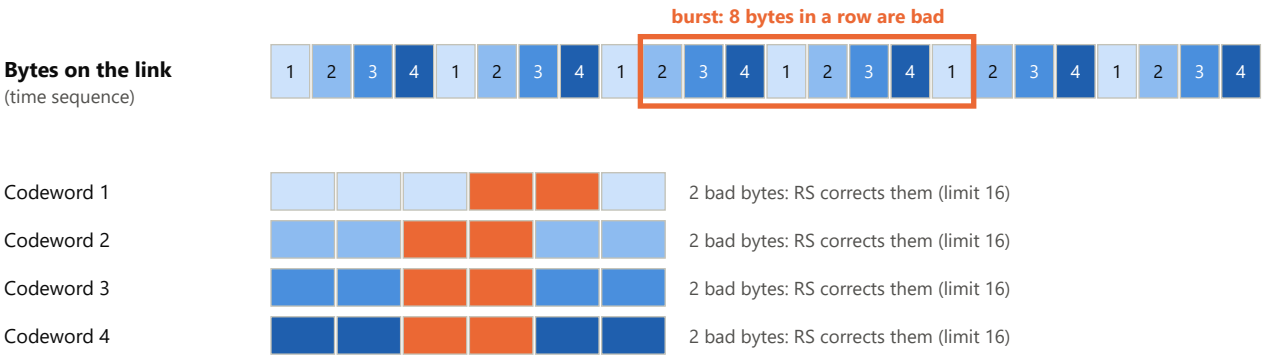


Figure 10: Interleaving spreads a burst of errors over four codewords

A burst of bad bytes in a row thus goes into four codewords. Each codeword can correct 16 bytes, so the HPDP can correct a burst of up to 4 × 16 = 64 bytes in a row. The four codewords with their parity make one codeblock of 4 × 255 = 1020 bytes.

12.3 Randomizer

The randomizer adds (XOR) a fixed pseudo-random sequence to the codeblock. The sequence starts with the bytes FF 48 0E C0 and repeats after 255 bytes. It starts again at each codeblock.

Why is this necessary? Some data has long runs of the same bit, for example an idle frame full of zeros. A receiver needs many changes between 0 and 1 to keep its clock. After the randomizer, the bits look random, and they have many changes. The ground does the same XOR again, which gives the original data back (XOR twice with the same value cancels).

12.4 The sync marker and the CADU

In front of each randomized codeblock, the HPDP puts the 4-byte attached sync marker (ASM): 1A CF FC 1D. The ASM is not randomized. The ground searches the stream for this pattern to find the start of each block. The result is the CADU (channel access data unit): 4 + 1020 = 1024 bytes.

> **NOTE**
> An advanced fact: an error pattern that repeats with a period that divides 255 is itself a valid RS codeword. The RS decoder then sees no error. In the HPDP, the frame header checks of the ground station catch such cases.

KEY POINTS

- RS(255,223) adds 32 parity bytes to 223 data bytes and corrects up to 16 bad bytes.
- Interleaving depth 4 lets the HPDP correct a burst of up to 64 bad bytes in a row.
- The randomizer gives the bit stream enough changes; the ground removes it with the same XOR.
- A CADU is the 4-byte sync marker 1ACFFC1D plus the 1020-byte codeblock.

Check your knowledge (the answers are in Appendix A)

1. How many bad bytes can RS(255,223) correct in one codeword?
2. Why does interleaving help against a burst of errors?
3. Why is the sync marker not randomized?

13 The error injector

13.1 Why add errors on purpose

The optical fiber on the desk is almost perfect: the measured bit error rate is less than 1 in 10^{12}. A real radio link has many more errors. To prove that the error correction works, the HPDP must have errors. The error injector adds them on purpose, between the channel coding and the link.

13.2 Modes

Table 12: Error injector modes

Mode	What it does
Off	No change to the data
Random	Changes random bits with a set bit error rate (BER), resolution 2^{-32}
Burst	Changes L bits in a row every N bytes
ASM	Changes k bits in each sync marker, to test the frame synchronizer

The start value (seed) of the random generator is a register, so a test can be repeated exactly. The injector counts each bit that it changes. This count is the true number of errors, and the tests compare the ground results with it.

13.3 Results: how far the code holds

Table 13: Bit error rate sweep over the fiber

Injected BER	Result at the ground (4 scenes)
0	4 of 4 scenes bit-exact
10^{-4}	1432 RS symbols corrected; 4 of 4 bit-exact
10^{-3}	14,508 RS symbols corrected; 4 of 4 bit-exact
2×10^{-3}	4 of 4 bit-exact
3×10^{-3}	1 uncorrectable codeblock; 3 of 4 bit-exact (only the scene with the bad codeblock is lost)
4×10^{-3} and more	0 of 4: scenes lost

The RS code thus holds up to approximately 2 errors in 1000 bits. At 3 in 1000, one codeblock fails, and only its own scene is lost, as requirement SYS-06 requires.

KEY POINTS

- The error injector makes a realistic noisy channel on a perfect fiber.
- It counts the errors it adds, so the tests can compare against the true number.
- The RS code corrected all errors up to a BER of 2×10^{-3}.

Check your knowledge (the answers are in Appendix A)

1. Why is a programmable seed useful?
2. What happened at a BER of 3×10^{-3}?

14 The optical link

14.1 The problems of a serial line

The fiber carries one bit after the other. There is no separate clock wire. The receiver must solve three problems:

- It must find the clock in the data. It can only do this if the data changes often between 0 and 1.
- The average signal must stay in the middle (DC balance), because the optical receiver cannot pass a constant level for a long time.
- It must find where each byte starts.

14.2 The 8b/10b code

The 8b/10b code solves all three problems. It changes each 8-bit byte into a 10-bit symbol. Of the 1024 possible 10-bit patterns, it uses only the patterns that have good properties:

- A symbol has five ones and five zeros, or six and four. A running disparity counter selects between two forms of each symbol, so the totals stay equal over time.
- There are never more than five equal bits in a row.
- Some extra symbols are control characters (K characters). K28.5 contains a **comma**: a bit pattern that no sequence of data symbols can make. The receiver uses it to find the start of each symbol.

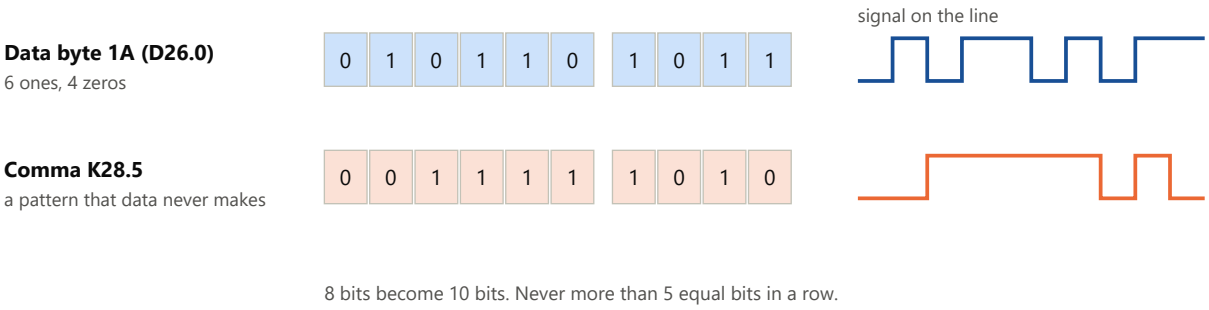


Figure 11: A data byte and the K28.5 comma as 10-bit symbols

14.3 The link transmitter

- It sends the CADU bytes as data characters, two bytes in each 16-bit word.
- When it has no data, it sends idle words: K28.5 followed by D16.2.
- At least every 2048 words (register CC_INTERVAL), it sends an idle word for clock correction (Section 14.5), also when data waits.

14.4 The GTH transceiver and the SFP module

The GTH transceiver is a fixed block in the FPGA. It does the 8b/10b coding and the serialization:

Worked example: the link rate

The link logic gives the GTH one 16-bit word each cycle of clk_link (62.5 MHz).

After 8b/10b, a 16-bit word is 20 bits. 20 bits × 62.5 MHz = 1.25 Gbaud.

One bit on the fiber thus lasts 800 picoseconds.

User payload: 16 bits × 62.5 MHz = 1000 Mbit/s. After the clock-correction words, at least 999 Mbit/s are available for CADUs.

The GTH gets its reference clock (156.25 MHz) from the Si570 oscillator on the board. The team measured it at 156,250,064 Hz. The SFP module changes the electrical bits into light pulses from an 850 nm laser. The PL drives the TX_DISABLE_B pins of the SFP cages high to switch the transmitters on.

14.5 The link receiver



Figure 12: The steps of the link receiver

1. SFP1 changes the light back into an electrical signal.
2. The clock and data recovery (CDR) circuit of the GTH finds the bit clock from the changes in the signal.
3. The deserializer makes 20-bit words. Comma alignment moves the word boundary until K28.5 is in byte 0.
4. The 8b/10b decoder changes each symbol back into a byte. It reports a code error if a symbol is not in the table or if the disparity is wrong.
5. The elastic buffer absorbs small differences between the recovered clock and the local clock. It removes or adds idle words to do this. This is clock correction.
6. The link receiver removes all K characters. Only the CADU bytes go on.

14.6 The lock state machine

The receiver passes data on only when it trusts the link. Its lock state machine declares LOCKED after 1024 words in a row with no code error. It loses lock if more than 8 code errors occur in 1024 words. It counts the code errors and the losses of lock.

14.7 The PRBS test

To measure the quality of the fiber link alone, the GTH has a PRBS-31 generator and checker. PRBS-31 is a pseudo-random bit sequence that repeats only after $2^{31}-1$ bits. The checker knows the sequence, so it can count each wrong bit.

Worked example: what "zero errors" proves

Test VER-06 ran PRBS-31 for 2700 seconds: 3.375×10^{12} bits with 0 errors.

With 0 errors in N bits, the BER is less than 3 / N with 95 % confidence.

$3 / (3.375 \times 10^{12}) = 8.9 \times 10^{-13}$. Thus BER < 8.9×10^{-13}.

14.8 The eye scan

An eye diagram shows many bits on top of each other. The open area in the middle is the “eye”. A large open eye means that the receiver can sample each bit with a good margin for noise and time errors. The In-System IBERT tool moves the sample point in time (horizontal) and in voltage (vertical) and measures the error rate at each point.

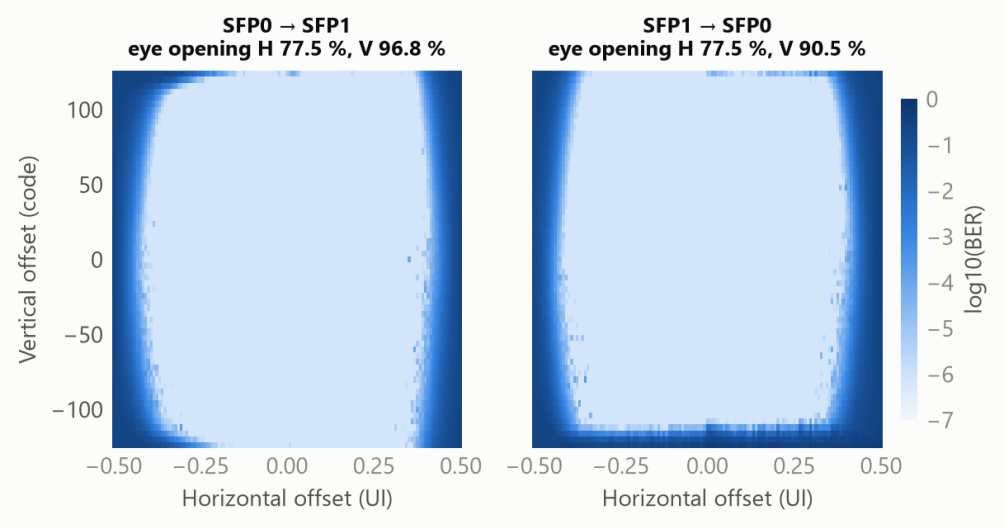


Figure 13: Measured eye scans in both directions of the fiber

From SFP0 to SFP1, the eye is open over 77.5 % of the bit time and 96.8 % of the voltage range. In the other direction, it is 77.5 % and 90.5 %. The fiber cable has two fibers, so both directions work.

KEY POINTS

- 8b/10b gives clock changes, DC balance and the K28.5 comma for word alignment.
- 20 bits × 62.5 MHz = 1.25 Gbaud; the payload is 1000 Mbit/s.
- The receiver recovers the clock, aligns on the comma, decodes, does clock correction and locks after 1024 clean words.
- PRBS-31: 3.375×10^{12} bits, 0 errors, BER < 8.9×10^{-13}. The eye is wide open.

Check your knowledge (the answers are in Appendix A)

1. Why does 8b/10b limit the number of equal bits in a row?
2. What is a comma, and why is it necessary?
3. When does the lock state machine declare LOCKED?
4. A PRBS test runs 10^{12} bits with 0 errors. What can you say about the BER?

15 The ground receiver

15.1 Frame synchronization

The link receiver gives a stream of bytes. The frame synchronizer must find the CADUs in this stream. It compares the stream with the sync marker 1ACFFC1D. It accepts a match with up to 2 wrong bits (register ASM_TOL), because the channel can damage the marker too.

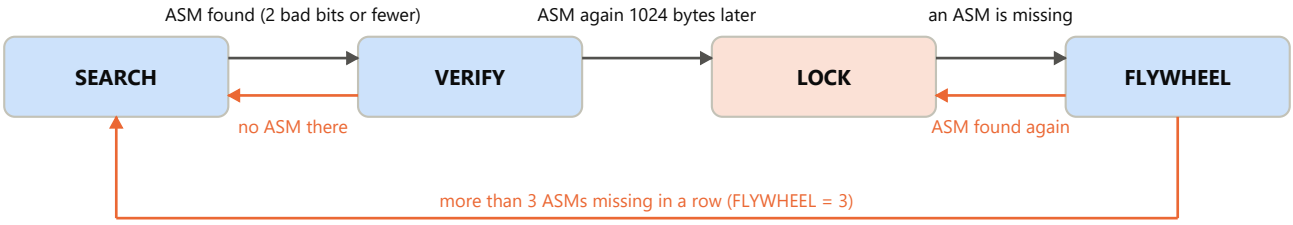


Figure 14: The states of the frame synchronizer

- **SEARCH:** look for the marker at each byte position.
- **VERIFY:** a marker was found. Check that the next marker is exactly 1024 bytes later. A random match in the data does not pass this check.
- **LOCK:** the synchronizer knows where each CADU starts.
- **FLYWHEEL:** one marker is missing, maybe because of errors. The synchronizer continues at the expected positions. After more than 3 missing markers in a row (register FLYWHEEL), it goes back to SEARCH.

15.2 Derandomizer and capture

The derandomizer does the same XOR as the randomizer, which gives the original codeblock. The capture block then adds a 4-byte status word to each 1020-byte codeblock. A DMA engine (S2MM direction) writes these 1024-byte records into a ring buffer in DDR4.

Table 14: The capture status word

Bits	Field of the status word
5 – 0	Number of wrong bits in the sync marker
7 – 6	Synchronizer state: 0 SEARCH, 1 VERIFY, 2 LOCK, 3 FLYWHEEL
15 – 8	Reserved
31 – 16	CADU sequence number

The ground receiver counts the CADUs, the losses of synchronization and the bit errors in the markers. It does not decode the Reed-Solomon code. The PC does that (Chapter 16). A hardware RS decoder was an optional item, and the team did not make it.

KEY POINTS

• The frame synchronizer finds the sync markers with up to 2 wrong bits and locks after a second marker 1024 bytes later.
• The flywheel keeps the lock through up to 3 missing markers.
• Each received codeblock goes to DDR4 as a 1024-byte record with a status word.

Check your knowledge (the answers are in Appendix A)

1. Why does the synchronizer need the VERIFY state?
2. What does the flywheel do?

16 The PC ground station

16.1 What it does

The ground station is the Python program sw/ground/hpdp_gs.py. It reads the received records and gets the image back. It does these steps:

1. Reed-Solomon decoding of each codeword, with the dual-basis conversion of the CCSDS standard. It corrects the errors and counts the corrections.
2. Frame checks: version, spacecraft ID and the frame counters. A gap in a counter shows a lost frame.
3. Removal of the idle frames.
4. Packet extraction with the first header pointer.
5. Scene assembly from the sequence flags and counters. A gap shows a lost packet.
6. CCSDS 123 decompression of each scene.
7. Comparison with the corrected scene of the Python model: pass or fail for each scene.
8. Display: a false-colour image from three bands, a single band, the spectrum of a pixel and the statistics.

16.2 Commands

Table 15: Ground station commands

Command	Use
decode	Decodes a capture file or a TCP stream. Options: --ref (reference for the comparison), --report, --out, --png, --crosscheck, --show
view	Shows decoded scenes
serve	Sends a capture file over TCP, to test the live input

16.3 How the data gets to the PC

In the board tests, the host script reads the records from the DDR4 memory through JTAG. The specification also asks for an upload over Ethernet (requirement SW-05). The board program has this function, and it builds. The team did not test it on the board, because the PC had no free Ethernet port. The project records this as a deviation.

16.4 Why the decoding is on the PC

The ground side of a real mission has large computers. Thus the team put the RS decoding and the decompression in Python, where they are easy to check. The RS decoder agreed with an independent library (reedsolo) on all 7088 codewords of a test. The cost is speed: approximately 3.4 ms for each codeblock and 2.5 s for each 128 × 64 × 64 scene.

> **NOTE**
> The reference for the comparison is the model's correction of the raw scene, not the clean scene before the reverse calibration. The correction rounds values, so only the model output is a bit-exact reference.

> **KEY POINTS**
> • The ground station corrects errors, checks frames, extracts packets, rebuilds scenes, decompresses and compares bit for bit.
> • In the board tests, the records come to the PC through JTAG; the Ethernet upload is a documented deviation.

Check your knowledge (the answers are in Appendix A)

1. How does the ground station find a lost frame?
2. Why is the model's corrected scene the reference, and not the clean scene?

Part C Advanced topics

Part C tells how the parts work together. It explains the control of the hardware, the clocks, the data streams and the protection against radiation. It also gives the method that the team used to prove that all of it is correct. It ends with the results and the procedures.

17 Registers and software

17.1 Registers

The PS controls the data chain through **registers**. A register is a 32-bit memory location in the PL that the processor can read or write through the AXI4-Lite bus. Settings (for example the image size) are registers that the PS writes. Counters and status flags are registers that the PS reads.

Table 16: The register address map (simplified)

Address	Block
0xA000_0000	System: ID, build information, global control, mode, interrupts
0xA001_0000 – 0xA004_0000	Sensor emulator, radiometric correction, corner turn, compressor
0xA005_0000 – 0xA007_0000	Packetizer, AOS framer and channel coding, error injector
0xA008_0000, 0xA009_0000	Link, ground receiver
0xA00F_0000	Radiation monitor: SEM, ECC and TMR counters
0xA010_0000, 0xA011_0000	DMA engines

Each block has the same first registers: ID at 0x00, CTRL at 0x04 (enable, soft reset, start), STATUS at 0x08 and two interrupt registers. The counters start at 0x80. They are 32 bits wide and they stop at the maximum value (they do not wrap). A write to CNT_CLEAR (0x7C) sets them to zero.

17.2 One source for hardware, software and documents

All registers are described in one file: regs/hpdp.rdl, in the SystemRDL language. From this one file, the PeakRDL tool makes the SystemVerilog register block, the C header, the Python definitions and an HTML description. Thus the hardware, the software and the documents always agree.

```
python regs/generate.py --html
```

17.3 The cfg_pending flag: a lesson from M4

The register block operates on clk_ctrl (100 MHz). The data chain operates on other clocks. A new setting crosses into the other clocks as a complete snapshot. This takes approximately one microsecond.

In M4, the system test found a fault. The program wrote the DMA route register and started the DMA at once. The new route had not arrived yet, so the first part of the calibration table went to the sensor emulator. The fix is the flag SYS.STATUS.cfg_pending. It stays 1 until each write has arrived in each clock domain. The program now waits for 0 before it starts a DMA transfer or a run.

17.4 The control program hpdp_run

hpdp_run is a bare-metal program: it runs directly on one A53 processor, with no operating system. It is built with AMD Vitis 2025.2. At each run, it does this sequence: reset, link start (wait for LOCKED), load the tables, write the settings, start.

The program has two ways to receive work:

- **Job blocks through JTAG** (all test runs). The host script makes a job block with the scenes, the tables and the settings. It loads the job, the program and the FPGA configuration through JTAG. The program reads the job at address 0x1000_0000 and writes a result block at 0x1000_1000. The host reads the results and checks them bit for bit. Each run is thus repeatable and checks itself.
- **The UART console with the SD card.** You type commands on a serial terminal (115200 bit/s). The program reads scenes and tables from a FAT32 SD card. The console makes the same job block, so both ways use the same checks.

Table 17: Main console commands

Command	What it does
ls, load, cal	List the SD card, load a scene file, load a calibration table
mode capture | downlink | both	Select where the compressed data goes
raw, correction, period, repeat	RAW mode, correction on or off, line period, repeats
ei off | ber | burst | asm, seed	Error injector settings
link, prbs	Link loopback settings, PRBS test
run, status, show	Start a run, show the counters, show the settings

17.5 The host scripts and a board run

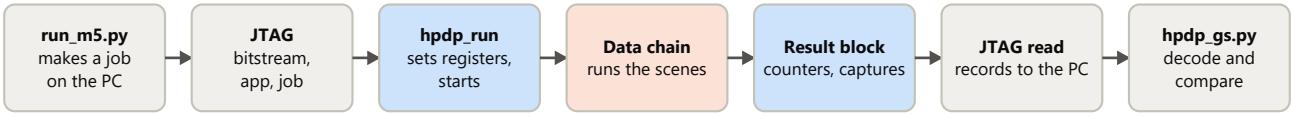


Figure 15: The sequence of a board run

sw/host/run_m5.py makes a full run of the chain. It generates or selects the scenes, runs the job, reads the results and checks them. Its main options are:

Table 18: Main options of run_m5.py

Option	Use
--work NAME	Folder for the run results (build/runs/NAME)
--size NX NY NZ D, --scenes N	Synthetic scene size and number of scenes
--scene FILE	Use a real scene file
--line-period N, --repeat N	Line timing; repeat the scenes for long runs
--ber X, --prbs N	Inject bit errors; run a PRBS test for N seconds
--loopback fabric | pma	Test the link without the fiber
--raw, --no-capture	RAW mode; do not capture the compressor output
--xsa FILE	Use another FPGA build, for example the 1024-pixel build
--wdt, --ecc, --sem-campaign	Radiation tests: watchdog, ECC injection, SEM upsets
--fake-board, --dry-run	Rehearse the run with no board

KEY POINTS

- The PS writes settings to and reads counters from registers through AXI4-Lite.
- One SystemRDL file generates the register hardware, the C and Python headers and the documents.
- Wait for cfg_pending = 0 after you write settings.
- hpdp_run takes job blocks through JTAG or commands from the UART console.

Check your knowledge (the answers are in Appendix A)

1. Why does the project generate the registers from one SystemRDL file?
2. What problem does cfg_pending solve?
3. Where does hpdp_run read its job, and where does it write the result?

18 Clocks, resets and clock-domain crossings

18.1 Why there are several clocks

Table 19: The clocks of the HPDP

Clock	Frequency	Used by	Why this clock
clk_sys	250 MHz	data path from DMA to ground receiver	fast enough for 1 sample each cycle with margin
clk_sensor	100 MHz	sensor emulator output	independent, like the oscillator of a real camera
clk_ctrl	100 MHz	registers, AXI4-Lite	the PS control bus
clk_link	62.5 MHz	link transmitter and receiver	set by the GTH: 1.25 Gbaud / 20
clk_fr	31.25 MHz	GTH helper logic, SEM, debug	free-running: it must run before the GTH clocks start

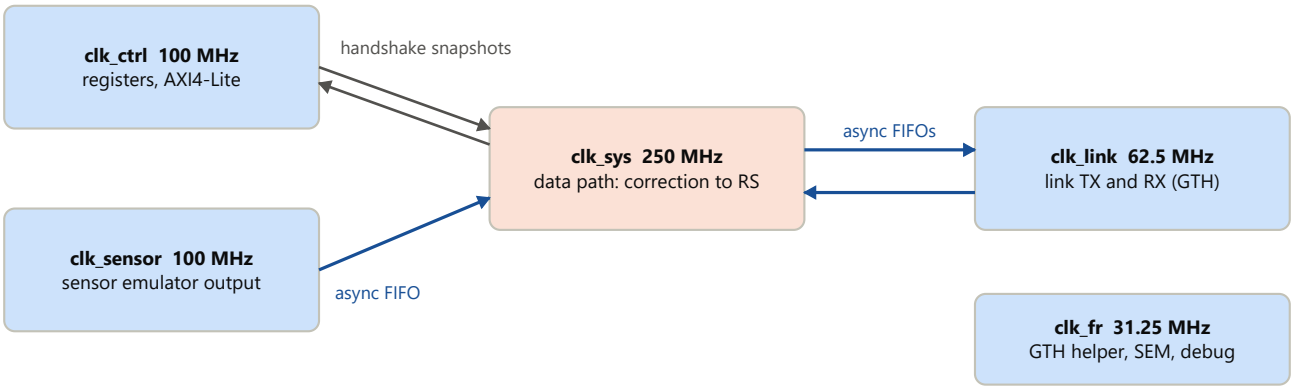


Figure 16: Clock domains and the crossings between them

18.2 The danger at a clock crossing

A flip-flop samples its input at the edge of its own clock. If the input changes at that moment, the flip-flop can stay between 0 and 1 for a short time. This is **metastability**. Also, a multi-bit value can arrive half old and half new. Thus a signal must never go from one clock domain to another without a special circuit.

The HPDP uses only the AMD XPM library circuits for this:

- **Asynchronous FIFOs** for data streams: sensor to data path, and data path to and from the link. A FIFO is a small memory with a write side and a read side on different clocks.
- **Handshake crossings** for register snapshots: a complete group of settings or counters crosses at one time, so it is never half old and half new.

The Vivado report_cdc check finds each crossing. The final build has 0 unsafe and 0 unknown crossings. One warning inside the AMD transceiver wizard has a written waiver with the reason.

18.3 Resets

Each clock domain has its own reset synchronizer. The reset starts at once (asynchronous) but ends at a clock edge (synchronous). Thus all flip-flops of a domain leave reset in the same clock cycle. The link logic stays in reset until the GTH reports that its own reset is complete.

18.4 Timing closure

At 250 MHz, a signal has 4 ns to go from one flip-flop through the logic to the next flip-flop. The tool calculates the time of each path. The **worst negative slack** (WNS) is the time that remains on the slowest path. A positive WNS means that all paths are fast enough.

Worked example: the final build m7h

WNS = +0.087 ns on clk_sys (250 MHz): the slowest path has 87 picoseconds of spare time.

WHS (hold) = +0.010 ns: no path is too fast either.

All 11 clocks meet timing. 36 bus-skew constraints on the clock crossings are met.

KEY POINTS

- Five clocks: 250 MHz data path, 100 MHz sensor and control, 62.5 MHz link, 31.25 MHz free-running.
- Data crosses clocks only through XPM asynchronous FIFOs or handshake snapshots.
- report_cdc: 0 unsafe, 0 unknown. Timing: WNS +0.087 ns at 250 MHz.

Check your knowledge (the answers are in Appendix A)

1. Why does the sensor emulator have its own clock?
2. What is metastability?
3. What does a WNS of +0.087 ns mean?

19 Data streams, back-pressure and overflow

19.1 The AXI4-Stream handshake

All blocks of the data path connect with AXI4-Stream. The sender drives the data (tdata) and a valid signal (tvalid). The receiver drives a ready signal (tready). One transfer occurs at each clock edge where tvalid and tready are both 1.

Table 20: AXI4-Stream signals

Signal	Meaning in the sample streams
tdata[15:0]	the sample, in the D low bits
tvalid	the sender has a sample
tready	the receiver can accept a sample
tuser[0]	first sample of a scene
tuser[1]	first sample of a line
tlast	last sample of a line

19.2 Back-pressure

When a receiver sets tready to 0, the sender must hold its data. This is **back-pressure**. Back-pressure is permitted after the corner turn. The compressor stops its full pipeline with one enable signal, and no data is lost. The sensor emulator does not accept back-pressure (Chapter 7).

Each block output has a register. Each tready path goes through a **skid buffer**. A skid buffer holds one extra word, so the tready signal can also have a register. This keeps the timing paths short at 250 MHz.

Each block was tested with random tvalid and tready patterns (requirement VER-01). For example, the skid buffer test passed 21,000 transfers with random stalls.

19.3 Where data can be lost, and how the HPDP reports it

Table 21: Loss cases and reports

Place	Cause	What the HPDP does
Sensor emulator	the next block is full	drops the sample, marks the line, counts it
Sensor emulator	DMA data late	sends zeros, counts an underrun
Corner turn	both line buffers full	sends a fill line, counts the dropped line
Scene guard	a scene is short	adds zeros to complete it; packets carry a flag
AOS framer	no science data ready	sends idle frames (no data lost)
Pipeline	no progress while input waits	the watchdog raises an interrupt

The rule is: a loss is allowed only if it is counted. The HPDP never loses data silently.

KEY POINTS

- A transfer occurs when tvalid and tready are both 1.
- Back-pressure is allowed after the corner turn, but never at the sensor emulator.
- Every loss of data is counted and reported.

Check your knowledge (the answers are in Appendix A)

1. When does a transfer occur on an AXI4-Stream interface?
2. What does a skid buffer do?

20 Protection against radiation

20.1 What radiation does to an FPGA

In space, high-energy particles go through the electronics. A particle can change the value of one memory bit or one flip-flop. This is a **single-event upset** (SEU). In an FPGA like the XCZU7EV, the configuration of the circuit is also in memory cells. An upset there can change the circuit itself.

The HPDP uses commercial parts, so it is not protected at the material level. It uses the same design methods as a real payload, in five layers.

20.2 Layer 1: configuration scrubbing

The AMD Soft Error Mitigation (SEM) IP reads the full configuration memory all the time. Each frame of the configuration memory has an error-correcting code. When SEM finds a single wrong bit, it writes the correct value back. One full scan of the 51,906 frames takes 154 ms.

On the board, the team injected 251 configuration upsets. SEM corrected 198 of them in 125 to 153 ms (the mean was 138 ms). The other 53 were in frames that the design does not use or that SEM masks. There were 0 uncorrectable upsets, and the scenes after the tests were bit-exact.

20.3 Layer 2: error-correcting memory (ECC)

A SECDED code adds check bits to each memory word. It corrects each single wrong bit (single-error correct) and finds each double wrong bit (double-error detect). All 56 UltraRAMs of the design use their built-in ECC. The compressor weights are in block RAM with ECC. When the calibration table reads a corrected entry, it writes the correct entry back, so that errors do not collect.

20.4 Layer 3: triple modular redundancy (TMR)

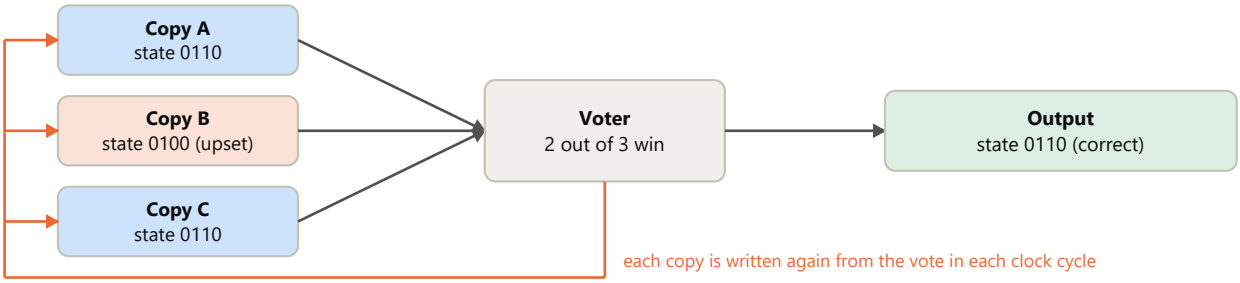


Figure 17: TMR: three copies and a majority voter

TMR keeps three copies of an important value. A voter selects the value that at least two copies agree on. Each copy is written again from the vote in each clock cycle, so an upset disappears after one cycle. A synthesis attribute (DONT_TOUCH) stops the tool from merging the three copies into one.

The HPDP uses TMR for the control state machines, the register block and the coder and neighbourhood memories of the compressor. Since build m7h, it also uses TMR for the address and count registers of the corner turn, the packetizer and the framer. For the small memories, a vote adds one logic level. A Hamming decoder would add five or six levels, which is too slow at 250 MHz.

20.5 Layer 4: watchdogs

- PL watchdogs are in the compressor, the downlink chain and the ground capture. If a block makes no progress for 2^{26} cycles (approximately 0.27 s) while data waits, it raises an interrupt.
- The PS system watchdog (10 s) resets the board if the program stops. A job flag arms it, so a debug session that halts the processor does not cause a reset. In the test, the program stopped on purpose and the board reset as expected.

20.6 Layer 5: flags in the data

Sometimes the HPDP finds an error that it cannot correct. Then it sets bit 0 of the flags in the packet headers of the scene and raises an interrupt. The ground station thus knows which scenes it must not trust. In a board test, a double-bit error in the calibration table caused all scenes of the run to have the flag.

20.7 The fault-injection campaign

How do you know that the protection works? The team changed random bits in a simulation of the design and classified each result:

Table 22: Fault classes

Result	Meaning
Masked	the changed bit had no effect on the output
Corrected	ECC or TMR corrected the bit
Detected	the design found the error and reported it (flag or counter)
Silent	the output was wrong and nothing reported it: the worst result

Table 23: Fault campaign results, final build m7h (1300 trials)

Area	Trials	Masked	Corrected	Detected	Silent
Protected state	820	15	567	238	**0**
Unprotected data registers	480	364	0	8	108 (22.5 %)

Zero silent results in 820 trials means that the silent fraction in the protected state is less than 0.4 %, with 95 % confidence. The unprotected data registers hold samples that pass through once. An upset there damages some

samples. The team accepted this as a residual risk.

20.8 How the campaign changed the design

The campaign on build m7g examined the address and count registers of the corner turn, the packetizer and the framer. Upsets there were silent 93 % to 100 % of the time. Build m7h added TMR to these registers, for 0.7 k more LUTs. After this change, all 220 simulated upsets in them were corrected.

> **NOTE**
> The campaign also found a fault in the test itself. In the simulator, the statement x[$urandom_range(...)] ^= 1 calculates the index two times, so it changed nothing in 462 of 1000 trials. The test now stores the index in a variable first. Always check that your test does what you think.

> **KEY POINTS**
> • SEM scrubbing corrected configuration upsets on the board in 125 to 153 ms.
> • ECC protects all UltraRAMs and the weights; TMR protects state machines, registers and control counters.
> • Watchdogs catch a stopped pipeline or program; flags mark scenes with uncorrectable errors.
> • The fault campaign found 0 silent results in 820 trials in protected state.

Check your knowledge (the answers are in Appendix A)

1. Why is an upset in the configuration memory of an FPGA dangerous?
2. What does SECDED mean?
3. Why is each TMR copy written again from the vote in each cycle?
4. Which result class is the worst in a fault campaign, and why?

21 How the team proved that it works

21.1 Model first

Before the team wrote any hardware, they wrote a Python model of each block. The model is bit-exact: it gives exactly the bits that the hardware must give. The hardware is then correct when its output is identical to the model output. One different bit shows a fault.

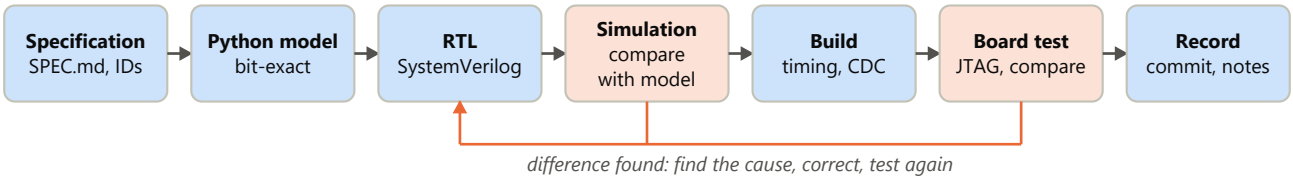


Figure 18: The method of work for each block

The compressor model can also record each intermediate value of each sample. When the hardware differs from the model, this record shows the exact sample and the exact pipeline stage where the difference starts.

21.2 The test levels

Table 24: The test levels

Level	What	Result
Python models	150 tests: compression round trips, RS, framing, frame sync, full chain	all pass
RTL regression	87 self-checking simulations with model data and random stalls; headless, approximately 9 minutes	all pass
Configuration matrix	all 864 compressor setting combinations (VER-02), 57 minutes	864 of 864 bit-exact

Level	What	Result
Corner cases	all zero, all maximum, random data, Nx = 1, minimum and maximum bands (VER-03)	all pass
System simulation	small scenes through the full chain with a link model and errors (VER-05)	bit-exact
Fault campaign	1300 random upsets	0 silent in protected state
Board	tests after each milestone, JTAG only	see Chapter 23

21.3 The milestones

Table 25: The project milestones

Milestone	Content	Done when (and result)
M0	Tool flow, base design, DMA loopback	64 MB loopback at 3865 MB/s
M1	Python models	bit-exact models of each block, with tests (150 today)
M2	Compressor hardware	bit-exact on all configurations, 250 MHz met
M3	Sensor emulator, correction, corner turn	bit-exact with the models, 250 MHz met
M4	First board run (compressor output to memory)	15 scenes bit-exact, 99.8 Msamples/s
M5	Downlink chain and optical link	PRBS, BER sweep and 10-minute rate test pass
M6	PC ground station	decode, display and verify; Ethernet upload a deviation
M7	Radiation protection	SEM, ECC, TMR, watchdogs; fault campaign; final build m7h

21.4 Problems that the team found, and the lessons

Table 26: Problems found during the project

Problem	Cause	Solution
The processor stopped at address 0xC after the first DDR4 write	The board preset assumed another memory module (x8 devices, not x16)	Correct DDR4 settings in the block design
The board started the flash image at each reset and damaged the loaded data	Boot mode pins set to QSPI flash	A temporary JTAG boot override at each session; the flash stays untouched
Vitis reported “invalid os”	A space in the folder path	Directory junctions with no spaces
The first table data went to the wrong block	A setting had not crossed the clock domain yet	The cfg_pending flag (Section 17.3)
Scenes merged under heavy overload	A short scene never ended	The scene guard (Section 9.4)
Idle frames passed as data at high BER	Exact match of the VCID	Match by bit distance (Section 11.4)
Long runs had underruns	The debugger halted the processor to read memory	Read memory with no halt
The fault test changed nothing in half of the trials	A simulator statement evaluated twice	Store the random index first

KEY POINTS

• A bit-exact Python model comes first; the hardware must match it bit for bit.
• 87 regression tests, the 864-configuration matrix, a fault campaign and board tests prove each requirement.
• Many important design features (cfg_pending, scene guard, TMR on the steering state) came from faults that the tests found.

Check your knowledge (the answers are in Appendix A)

1. Why is a bit-exact model so useful for hardware tests?
2. How many compressor configurations did the team test, and with which result?

22 Implementation results

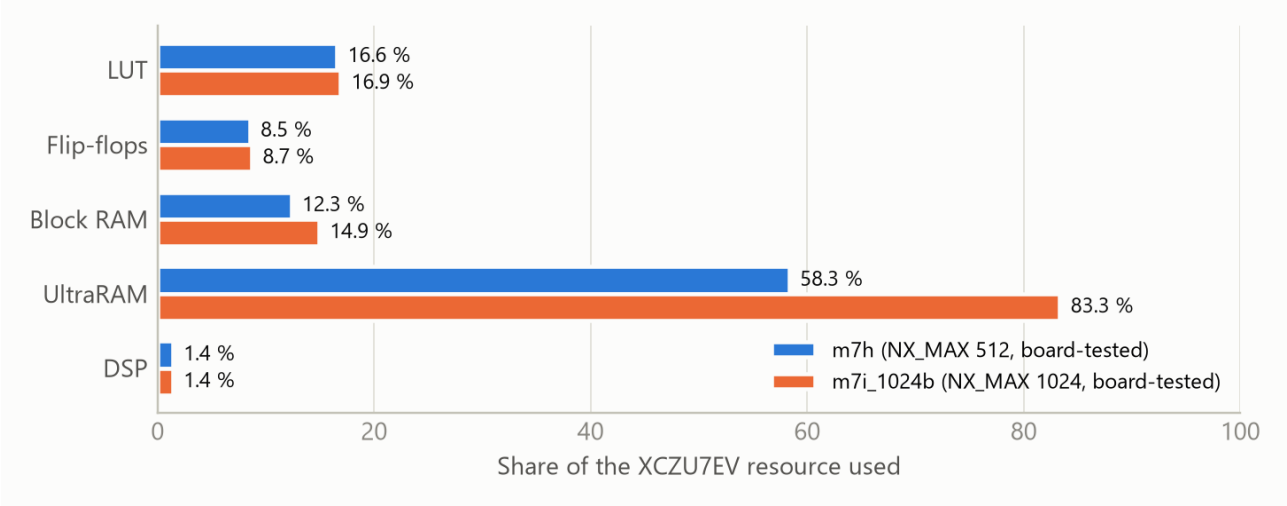


Figure 19: FPGA resources used by the two final builds

Table 27: Resources and timing of the two final builds

Resource	Final build m7h (512 pixels)	Build m7i_1024b (1024 pixels)
LUT	38.2 k (16.6 %)	38.9 k
Flip-flop	39.2 k (8.5 %)	40.1 k
Block RAM (36 Kb)	38.5 (12.3 %)	46.5
UltraRAM	56 (58.3 %)	80 (83.3 %)
DSP	24 (1.4 %)	24
GTH transceivers	2	2
Setup slack (WNS) at 250 MHz	+0.087 ns	+0.012 ns

The design uses less than one fifth of the logic. The largest user of a resource is UltraRAM: the calibration table, the corner-turn buffers and the compressor line buffer with ECC.

22.1 The 1024-pixel build

Requirement SYS-04 says that the design must scale to 1024 pixels by a parameter. With one table entry in each memory word, the calibration table would need 64 UltraRAMs. The design would then need 112 UltraRAMs, but only 96 exist. The solution is a second table layout: two entries (dark and gain) in each word, and the bad-pixel flags in a separate block RAM. The build selects this layout automatically. The 1024-pixel build uses 80 UltraRAMs and meets 250 MHz. The team tested it on the board with a 1024-pixel Hyperion scene. A build for 1024 × 512 would need 160 UltraRAMs and does not fit.

KEY POINTS

- The final build uses 16.6 % of the LUTs and 58.3 % of the UltraRAMs and meets 250 MHz.
- A packed table layout lets the 1024-pixel build fit in 80 of 96 UltraRAMs.

Check your knowledge (the answers are in Appendix A)

1. Which resource limits the size of the HPDP?

23 Results summary

Table 28: The main results

Area	Test	Result
Throughput	Mission rate for 10 minutes (VER-08)	600.1 s at 76.8 Msamples/s, 2344 scenes, 0 dropped lines, 0 sync losses
Throughput	Compressor capacity	250 Msamples/s (1 sample each clock at 250 MHz)
End to end	Scenes over the fiber at BER 0, 10^{-4}, 10^{-3} (VER-07)	all bit-exact
End to end	BER sweep	all corrected up to 2×10^{-3}
Compression	63 real scenes	all lossless; 4.4 to 6.0 bits per sample on average (2.3:1 to 3.1:1)
Link	PRBS-31 over the fiber (VER-06)	3.4×10^{12} bits, 0 errors, BER < 8.9×10^{-13}
Link	Eye scan (LNK-10)	77.5 % × 96.8 % open (SFP0 to SFP1)
RAW mode	SYS-09	99.1 % of lines dropped and counted, as required
Radiation	SEM upsets on the board	198 corrected in 125 to 153 ms, 0 uncorrectable
Radiation	Fault campaign	0 silent in 820 protected trials
Timing	All clocks (VER-09)	met; WNS +0.087 ns; report_cdc clean
Requirements	102 v1.0 shall/should items	99 met, 1 partly met (SW-05), 2 pending (O-5)

KEY POINTS

• The HPDP meets the mission rate, gets every test scene back bit-exactly and survives injected upsets.

24 Status, limits and next steps

24.1 Status

- 99 of the 102 v1.0 requirements are met. None is unmet.
- **SW-05 is partly met.** The Ethernet upload of the received data is written and builds. It is not tested on the board, because the PC had no free Ethernet port. The data goes to the PC through JTAG. A USB-Ethernet adapter and approximately 10 minutes would close this item.
- **CMP-02 and MDL-02 are pending** (open item O-5). They need a check of the compressor output with an independent CCSDS 123 decoder: ESA WhiteDwarf 2.0 or the CNES tools. Both need a user account.

24.2 Limits

- Commercial parts, no radiation qualification.
- No radio link and no real camera.
- Lossless mode only. The near-lossless mode of the standard is not in v1.0.
- RS decoding and decompression are in Python on the PC, so they are slow.
- Upsets in unprotected data registers are silent approximately one time in four (accepted residual risk).

24.3 Possible next steps

- Decode the test data with an independent CCSDS 123 decoder (closes O-5).
- Test the Ethernet upload with an adapter (closes the SW-05 deviation).
- Near-lossless compression and the hybrid entropy coder.
- A hardware RS decoder, a faster ground decoder (C or numba), and Linux in place of bare-metal.
- A return channel: the fiber cable has two fibers, so this is now possible.

> **KEY POINTS**
> • The project is complete except one deviation (Ethernet upload) and one external check (independent decoder).

25 Procedures

This chapter gives the procedures to repeat the main tests. Do the procedures in the project folder. The commands are for the Windows command line with Vivado and Vitis 2025.2 installed.

25.1 Prepare the board

> **CAUTION**
> Use JTAG only. Do not write the QSPI flash of the board. The test flow loads all files into volatile memory.
> Do not bend the fiber tightly. Put the dust caps on the SFP modules and the fiber ends when you disconnect them.

1. Connect the USB cable (JTAG and UART) between the board and the PC.
2. Connect the OM3 fiber between SFP0 and SFP1.
3. If you use the UART console, put the SD card with the scene files into the board.
4. Set the power switch of the board to on.

25.2 Run the model tests

1. Open a command window in the project folder.
2. Type the command below and push Enter.
3. Make sure that all 150 tests pass.

```
python -m pytest model
```

25.3 Run the hardware regression

1. Type the command below. The regression takes approximately 10 minutes.
2. Make sure that all 87 tests show PASS.

```
vivado -mode batch -source tb/run_sim.tcl
```

25.4 Build the FPGA configuration

1. Set HPDP_BUILD_DIR to a new folder in build/ if you want to keep the old build.
2. For the 1024-pixel build, set HPDP_NX_MAX to 1024.
3. Type the command below.

4 Make sure that the exit code is 0. The script returns 1 if the timing is not met.

```
vivado -mode batch -source vivado/build.tcl
```

25.5 Build the program

```
vitis -s sw/baremetal/build.py build/m7h/hpdp.xsa
```

25.6 Do a board run

1 Type the command below. Use a new name for each run.

2 Wait until the script shows the result.

3 Make sure that the result is PASS. The run files are in build/runs/NAME.

```
python sw/host/run_m5.py --work NAME
```

25.7 Decode the run at the ground station

```
python sw/ground/hpdp_gs.py decode build/runs/NAME/gr.bin --ref build/runs/NAME --show
```

25.8 If a fault occurs

Table 29: Faults and actions

Symptom	Possible cause	Action
The link does not lock. LNK.STATUS shows 0xF07 and many code errors	The fiber is not connected	Connect the fiber. To test the board without the fiber, use --loopback pma
The processor stops at address 0xC	Wrong DDR4 settings for the memory module	Use the block design of the project, which has the correct settings
The loaded program disappears after a reset	The board starts from the QSPI flash	Let the host script set the JTAG boot override; do not change the flash
Vitis reports “invalid os”	A space in the path	Use the junction folders (docs/new_system_setup.md)
Underruns in long runs	The processor was halted to read memory	Use the read method that does not halt the processor

Appendix A Answers to the check questions

Chapter 2: The HPDP in one page

1. Why must a satellite compress its image data?
The camera makes more data than the link can send. In the HPDP reference mission, the camera makes 921.6 Mbit/s, but the link carries only approximately 857 Mbit/s of user data.

2. What does “bit-exact” mean?
The image that the ground station rebuilds is identical to the original corrected image. Not one bit is different.

3. Which part of a real satellite does the optical fiber replace?
The radio link between the satellite and the ground station.

Chapter 3: Basic ideas

1. How many values can a 12-bit sample have?
4096 values (0 to 4095), because 2 to the power 12 is 4096.

2. The clock is 250 MHz. How long is one clock cycle?
4 nanoseconds (1 divided by 250,000,000).

3. How many samples does one HPDP scene have?
512 × 256 × 150 = 19,660,800 samples.

4. Why is it a problem when the electronics cannot accept a line from a pushbroom camera?
The camera cannot wait. The ground moves on, so the line is lost and cannot be recorded again.

Chapter 4: The hardware

1. What is the difference between the PS and the PL?
The PS is the processor part (four Arm A53 processors). It runs the control program. The PL is the FPGA part. It contains the data chain circuit.

2. Which memory type is the largest on-chip memory, and how many does the XCZU7EV have?
UltraRAM (288 kilobits each). The XCZU7EV has 96.

3. What does an SFP module do?
It changes electrical signals into light (transmitter) and light into electrical signals (receiver).

Chapter 5: The data chain

1. Which block puts the compressed bits into packets?
The space packetizer.

2. How long is one CADU?
1024 bytes: a 4-byte sync marker and a 1020-byte codeblock.

3. Which program does the decompression?
The ground station program on the PC (hpdp_gs.py), written in Python.

Chapter 6: The data budget

1. Why is the user data capacity (857 Mbit/s) less than the link payload (1000 Mbit/s)?
Part of each CADU is not user data: the sync marker, the Reed-Solomon parity and the frame headers (CADU efficiency 86.3 %). The packet headers take a small part more (99.3 %).

2. What percentage of the link does a scene with 2:1 compression use?
460.8 / 857 = approximately 54 %.

3. Which mode shows that raw data does not fit, and what did the board test show?
RAW mode. At the mission rate, 99.1 % of the lines were dropped and counted.

Chapter 7: The sensor emulator

1. Which value of LINE_PERIOD gives 1000 lines each second?
100,000, because clk_sensor is 100 MHz and one line must start each millisecond.

2. What does the emulator do when the next block cannot accept a sample?
It drops the sample, marks the line and counts the event. It does not wait.

3. What is an underrun?
The DMA did not deliver the data of a line in time. The emulator sends zeros and counts the event. It is a fault of the test setup.

Chapter 8: Radiometric correction

1. A sensor gives a value when there is no light. What is the name of this error?
Dark offset.

2. Which gain value in the table means "no change"?
0x4000 (16384), which is 1.0.

3. x = 500, dark = 100, gain = 0x2000. What is c?
0x2000 is 0.5. (500 − 100) × 8192 = 3,276,800; + 8192 = 3,284,992; >> 14 = 200. Check: 400 × 0.5 = 200.

4. A bad pixel has a bad left neighbour and a good right neighbour (value 700). What is the new value?
700, the value of the right neighbour.

Chapter 9: Corner turn

1. In BIP order, which sample comes after pixel 0, band 0?
Pixel 0, band 1.

2. Why does the corner turn need two line buffers?
It must write a new line while it reads the previous line in a different sequence. One buffer receives while the other buffer sends.

3. Why does the HPDP send a fill line in place of a dropped line?
Each scene must keep its 256 lines. Without a fill line, the following lines move and the compressor loses the scene boundaries.

Chapter 10: CCSDS 123

1. Why can the decoder make the same prediction as the encoder?
The prediction uses only samples that come before the current sample. The decoder has already decoded these samples exactly.

2. Which neighbours does the predictor use in the current band?
W, NW, N and NE.

3. Why does BIP order help the hardware to run at 250 MHz?
The state of a band is necessary again only Nz cycles later. The circuit has Nz cycles to update it, so the pipeline can be deep without hazards.

4. Write the code for the number 9 with k = 2.
9 >> 2 = 2, thus two zeros and then a one. Then come the 2 low bits of 9 (binary 01): 0 0 1 01, which is 5 bits.

5. What happens when you set P = 0?
The predictor does not use earlier bands. Real scenes then need 12 % to 67 % more bits.

Chapter 11: Packets and frames

1. Why is the AOS frame 892 bytes long?
892 = 223 × 4. One frame fills one Reed-Solomon codeblock of four interleaved codewords, each with 223 data bytes.

2. What does the first header pointer value 0x7FF mean?
No packet starts in this frame.

3. Why does the framer send idle frames?
The downlink must never stop, so that the receiver stays synchronized. Idle frames fill the time when no science data is ready.

Chapter 12: Channel coding

1. How many bad bytes can RS(255,223) correct in one codeword?
16.

2. Why does interleaving help against a burst of errors?
Consecutive bytes go to different codewords. A burst thus divides over four codewords, and each codeword gets only a quarter of the bad bytes.

3. Why is the sync marker not randomized?
The ground must find it in the raw stream before it can remove the randomization. A fixed, known pattern is necessary for the search.

Chapter 13: The error injector

1. Why is a programmable seed useful?
The same seed gives the same error pattern, so a failed test can be repeated exactly.

2. What happened at a BER of 3×10^{-3}?
One codeblock was uncorrectable. Only the scene that contained it was lost; the other three scenes were bit-exact.

Chapter 14: The optical link

1. Why does 8b/10b limit the number of equal bits in a row?
The receiver finds the clock from the changes between 0 and 1. Long runs with no change would let the recovered clock drift.

2. What is a comma, and why is it necessary?
A bit pattern (in K28.5) that data symbols never make. The receiver uses it to find where each symbol and word starts.

3. When does the lock state machine declare LOCKED?
After 1024 words in a row with no code error.

4. A PRBS test runs 10^{12} bits with 0 errors. What can you say about the BER?
BER < 3 / 10^{12} = 3×10^{-12}, with 95 % confidence.

Chapter 15: The ground receiver

1. Why does the synchronizer need the VERIFY state?
The 4-byte pattern can occur by chance in the data. A second marker exactly 1024 bytes later confirms that the first one was a real marker.

2. What does the flywheel do?
It keeps the lock when a marker is damaged, and it continues at the expected positions for up to 3 missing markers.

Chapter 16: The ground station

1. How does the ground station find a lost frame?
The frame counter of the virtual channel has a gap.

2. Why is the model's corrected scene the reference, and not the clean scene?
The correction rounds and clamps values. Only the output of the bit-exact model is identical to what the hardware makes.

Chapter 17: Registers and software

1. Why does the project generate the registers from one SystemRDL file?
So that the hardware, the C program, the Python scripts and the documents always use the same addresses and fields.

2. What problem does cfg_pending solve?
A setting needs approximately 1 µs to reach the other clock domains. Without the flag, the program can start a DMA transfer before the new setting is active.

3. Where does hpdp_run read its job, and where does it write the result?
Job at 0x1000_0000, result block at 0x1000_1000.

Chapter 18: Clocks

1. Why does the sensor emulator have its own clock?
A real camera has its own oscillator, independent of the processor. The separate clock makes the test realistic, including the clock crossing.

2. What is metastability?
A flip-flop that samples a changing input can stay between 0 and 1 for a short time. It can cause wrong values at a clock crossing.

3. What does a WNS of +0.087 ns mean?
The slowest path of the design has 0.087 ns of spare time. All paths meet the 4 ns limit.

Chapter 19: Data streams

1. When does a transfer occur on an AXI4-Stream interface?
At a clock edge where tvalid and tready are both 1.

2. What does a skid buffer do?
It holds one extra word, so the tready signal can have a register and the timing path stays short.

Chapter 20: Radiation

1. Why is an upset in the configuration memory of an FPGA dangerous?
The configuration memory defines the circuit. An upset can change the circuit itself, not only one data value.

2. What does SECDED mean?
Single-error correct, double-error detect.

3. Why is each TMR copy written again from the vote in each cycle?
So that an upset in one copy disappears after one cycle and cannot add to a second upset later.

4. Which result class is the worst in a fault campaign, and why?
Silent: the output is wrong and nothing reports it, so nobody knows that the data is bad.

Chapter 21: Verification

1. Why is a bit-exact model so useful for hardware tests?
The hardware output must be identical to the model output. Any difference is a fault, and the model trace shows where it starts.

2. How many compressor configurations did the team test, and with which result?
All 864 configurations of the VER-02 matrix; all 864 were bit-exact.

Chapter 22: Implementation

1. Which resource limits the size of the HPDP?
UltraRAM: the 1024-pixel build already uses 80 of the 96 UltraRAMs.

Appendix B Numbers to remember

Table 30: Numbers to remember

Item	Value
Image	512 pixels × 150 bands, 12-bit samples, 1000 lines each second
Raw rate	76.8 Msamples/s = 921.6 Mbit/s
Scene	256 lines = 19,660,800 samples
Data path clock	250 MHz (4 ns); 1 sample each clock
Calibration	c = clamp(((x − dark) × gain + 2^13) >> 14, 0, 2^D − 1); gain 0x4000 = 1.0
Compressor	P = 3 earlier bands (up to 15), Ω = 13, U_{max} = 18; NZ_MIN = 16
Real compression	2.3:1 to 3.1:1 on average; all lossless
Space Packet	14-byte header + 2048 data bytes
AOS frame	892 bytes = 6 + 2 + 884; idle frames on VCID 63
Reed-Solomon	RS(255,223), corrects 16 bytes per codeword; I = 4; codeblock 1020 bytes
Randomizer	starts FF 48 0E C0; period 255 bytes
Sync marker, CADU	1A CF FC 1D; CADU = 1024 bytes
Link	1.25 Gbaud, 8b/10b, 62.5 MHz × 16 bits, 1000 Mbit/s payload, 850 nm, OM3
Link capacity for user data	≈ 857 Mbit/s
Link lock	1024 clean words; lost after more than 8 code errors in 1024 words
Frame sync	up to 2 wrong ASM bits; VERIFY 1024 bytes later; FLYWHEEL 3
Capture record	1020-byte codeblock + 4-byte status word
SEM	full scan 154 ms; upsets corrected in 125 to 153 ms
Final build m7h	38.2 k LUT, 56 URAM, 24 DSP, WNS +0.087 ns

Appendix C Where things are in the repository

Table 31: The repository folders

Folder	Contents
SPEC.md	The specification: all requirements, interfaces and milestones
rtl/frontend/	Sensor emulator, radiometric correction, corner turn, scene guard
rtl/ccsds123/	The compressor: neighbourhood, local differences, predictor, coder, packer
rtl/downlink/	Space packetizer, AOS framer, RS encoder, CADU framer, error injector
rtl/link/	Link layer, link transmitter and receiver
rtl/ground/	Frame synchronizer with derandomizer, ground capture
rtl/common/	Skid buffer, clock-crossing bus, ECC and TMR memories, FIFO, watchdog
rtl/rad/	SEM controller support
rtl/system/	hpdp_core (data path, registers, crossings), hpdp_top (transceivers and block design)
rtl/regs/, regs/	The generated register block; the SystemRDL source and its generator
model/	Bit-exact Python models, their tests, hpdp_tool.py and hpdp_datasets.py
tb/	Testbenches and test-vector generators; tb/run_sim.tcl runs the regression
vivado/	Build scripts, block design, constraints and IP settings
sw/baremetal/	The A53 program hpdp_run (job, console, network) and its build script
sw/host/	Host scripts: run_m4.py, run_m5.py, sd_put.py, eye-scan tools
sw/ground/	The ground station hpdp_gs.py and its tests
docs/	Milestone notes, project report, real-data results, runbooks, these PDF documents
build/	Builds, reports and run results (not in version control)

Appendix D Technical names and abbreviations

Table 32: Technical names and abbreviations

Name	Meaning
AOS	Advanced Orbiting Systems: the CCSDS protocol for transfer frames
APID	Application process identifier: the source field of a Space Packet
ASM	Attached sync marker, 1A CF FC 1D: marks the start of each CADU
AXI	The on-chip bus standard of AMD devices (AXI4-Lite for registers, AXI4-Stream for data)
BER	Bit error rate: wrong bits divided by all bits
BIL, BIP, BSQ	Band-interleaved by line, by pixel; band-sequential: sample sequences
BRAM, URAM	Block RAM (36 Kb) and UltraRAM (288 Kb): on-chip memories
CADU	Channel access data unit: ASM plus randomized codeblock (1024 bytes)
CCSDS	Consultative Committee for Space Data Systems: writes the space data standards
CDC	Clock-domain crossing
CDR	Clock and data recovery: finds the bit clock in a serial signal
DMA	Direct memory access: moves data between memory and the PL without the processor
DSP	A hardware multiplier and adder in the FPGA
ECC, SECDED	Error-correcting code; single-error correct, double-error detect
FHP	First header pointer of an AOS frame
FIFO	First in, first out: a memory that keeps the sequence of the data
FPGA	Field-programmable gate array
FSM	Finite state machine
GTH	The multi-gigabit transceiver of the device
JTAG	The debug and programming connection of the board
LFSR, PRBS	Linear feedback shift register; pseudo-random bit sequence
LUT	Lookup table: the basic logic part of the FPGA
PS, PL	Processing system (Arm processors); programmable logic (FPGA part)
RS	Reed-Solomon error-correcting code
SEM	Soft Error Mitigation IP: scrubs the configuration memory
SEU	Single-event upset: a bit changed by radiation
SFP	Small form-factor pluggable optical module
TMR	Triple modular redundancy: three copies and a majority voter
VCID	Virtual channel identifier of an AOS frame (63 = idle)
WNS, WHS	Worst negative slack (setup) and worst hold slack: timing margins
XPM	AMD macro library, used for the clock-domain crossings